

Society for Computer Technology & Research's

Pune Institute of Computer Technology

Department of Information Technology

LAB MANUAL

DOSSL

Class: T. Y. B-Tech

SCRT's Pune Institute of Computer Technology, Pune

Department of Information Technology



Lab Manual

Class: T.Y. B.Tech.

Semester: I

Course Code: 2635PE34

Course Name: DevOps for Scalable Systems Lab

Prepared By:

Mr. Abhinay Dhamankar

Mrs. Amruta Patil

Mr. Jagdish Kamble

Compiled By:

Mr. Abhinay Dhamankar

IMPORTANT GUIDELINES

The laboratory assignments are to be submitted by students in the form of journals. The Journal consists of a table of contents (Index), and a handwritten write-up of each assignment (Title, Aim, Problem Statement, Outcomes, Date of Implementation, Date of Submission, assessor's sign, Theory- Concept written using coding standards, sample test cases, etc.)

1. The examination will be based on ORAL questions derived from the assignments conducted during the course, along with various applications and relevant theoretical concepts related to the subject.
2. The candidate is expected to have a thorough understanding of the theoretical principles underlying the experiments.
3. The examination shall be conducted only after the candidate's journal has been completed in all aspects and duly certified by the concerned faculty and the Head of the Department.

Guidelines for Lab /TW Assessment.

1. Examiners shall evaluate the term work based on the student's performance, considering parameters such as timely completion of practical assignments, the methodology used for their implementation, prompt submission of handwritten reports with corresponding results, attendance, and overall discipline.
2. Examiners will assess the student's understanding of the practical performed by asking relevant questions pertaining to the theoretical concepts and the implementation of the experiments undertaken.
3. The concerned faculty member(s) must verify the student's proficiency in using relevant software and hardware tools, including compilers, debuggers, coding standards, and the implementation of the required algorithms.

2635PE34: DEVOPS FOR SCALABLE SYSTEMS LABORATORY (DOSSL)

COURSE PLAN : DevOps for Scalable Systems Laboratory			
Main Category	Program Courses	Sub- Category	Program Elective Course (PEC)
Class	TY-9, 10, 11	Semester	V
Course Code	2635PE34	Course Name	DevOps for Scalable Systems Laboratory
Teaching scheme		Evaluation Scheme	
Practical	2 Hrs. / week	Continuous Internal Evaluation (TW)	25 Marks
Credits	1	End Sem Examination (OR)	25 Marks
Continuous Internal Evaluation (CIE)		Based on the Assignments and mock orals	

Course Objectives:

The objective of the course is to provide students with

- Introduction of version control, continuous integration, and build automation using Git and Jenkins.
- Practical knowledge of containerization and application orchestration with Docker and Kubernetes.
- Designing and implementing automated CI/CD pipelines with integrated testing.
- Monitoring and visualizing application performance using Prometheus and Grafana

Course Outcomes: After completion of this course, students will be able to:	BTL
CO-1 : Apply version control and build automation by creating Git repositories and configuring Jenkins jobs.	L3 – Apply
CO-2 : Implement containerization with Docker and deploy scalable applications on Kubernetes.	L3 – Apply
CO-3 : Design Jenkins pipelines with automated testing to ensure code reliability and quality.	L6 – Create
CO-4 : Analyse and visualize application performance metrics using Prometheus and Grafana dashboards.	L4 – Analyze

CO-PO Articulation Matrix:

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO1	PSO2
CO-1	3	2	2	-	3	-	-	2	1	1	2	3	2
CO-2	3	2	3	2	3	1	-	2	-	2	2	3	2
CO-3	3	3	3	2	3	-	-	2	1	2	2	3	2
CO-4	2	3	2	3	3	1	-	1	2	1	3	3	1
Average	2.7	2.5	2.5	1.7	3.0	0.5	0.0	1.7	1.0	1.5	2.2	3.00	1.75

Prerequisites: Students should have prior knowledge of Basic programming and familiarity with software development concepts.

PRACTICAL / ASSIGNMENTS PLAN			
Assignment No.	Name of the Assignment	COs Mapped	No. of Hrs.
1	Git Basics: Create a GitHub repository for a sample application, commit code, and demonstrate basic Git operations (clone, branch, merge, push, pull).	CO1	06
2	Git Branching & Collaboration: Implement branching strategies (feature branches, pull requests, and merging) to simulate collaborative development in a distributed team.	CO1	06
3	Jenkins Build: Configure Jenkins to manually pull the sample application from GitHub and build it successfully.	CO1	06
4	Dockerization: Containerize the given sample application using Docker so that it runs seamlessly on any machine without manual setup.	CO2	06
5	Kubernetes Deployment: Deploy the Dockerized application on Kubernetes (using Minikube), configure scaling, and ensure it is accessible to users.	CO2	08
6	Jenkins Pipeline with Automated Testing: Integrate pre-written test scripts into a Jenkins pipeline to ensure that the build fails automatically if any test cases fail.	CO3	04
7	Monitoring with Prometheus: Set up Prometheus to collect and monitor application performance metrics in real time.	CO4	04
8	Visualization with Grafana: Create Grafana dashboards to visualize and analyze the real-time metrics collected by Prometheus.	CO4	04

This is for Student's Journal
Assignment write-up should be in this format

Date:

Assignment No:

Title:

Aim:

Objective:

Problem Statement:

Theory / Procedure / Diagrams:

Algorithm / Methods / Steps: (if applicable)

Print Outs :

Conclusion:

Assignment No: 1

Assignment No: 1

Title

Git Basics and Version Control for a Sample Web Application

Course Outcome (CO) Mapped: CO1

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 04

1. Aim

To create a GitHub repository for a sample web application and demonstrate basic Git version control operations, including repository creation, cloning, committing changes, branch creation, merging, pushing, and pulling code.

2. Problem Statement

Develop a simple **Student Task Manager Web Application** and manage its source code using Git and GitHub.

The application should provide a basic web interface to display and manage student tasks.

Perform the following version control operations:

1. Create a GitHub repository for the Student Task Manager Web Application.
2. Clone the remote GitHub repository to the local system.
3. Add the sample web application source code to the local repository.
4. Stage and commit the application files using Git.
5. Push the committed source code to the GitHub repository.
6. Create a new branch for application modifications.
7. Modify the web application in the newly created branch.
8. Commit the changes made in the branch.
9. Merge the branch with the main branch.
10. Pull the latest changes from the remote GitHub repository.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand the fundamentals of version control systems.
- Create and manage a Git repository.

- Use Git for tracking source code changes.
- Perform basic Git operations such as clone, add, commit, push, and pull.
- Create and manage branches in Git.
- Merge changes from one branch into another.
- Manage source code using a remote GitHub repository.
- Apply version control practices to a web application project.

4. Prerequisites

Students should be familiar with:

- Basic HTML, CSS, and JavaScript.
- Basic command-line operations.
- Basic understanding of software development.
- Basic knowledge of web applications.
- GitHub account creation and login.

5. Software and Hardware Requirements

Software

- Git
- GitHub Account
- Visual Studio Code or any suitable code editor
- Google Chrome / Microsoft Edge / any modern web browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 4 GB RAM
- Internet connectivity
- Minimum 1 GB free disk space

6. Theory

6.1 Version Control System

A Version Control System (VCS) is a software tool used to track and manage changes made to source code and project files.

It enables developers to maintain different versions of a project and collaborate effectively.

Benefits of Version Control Systems include:

- Tracking source code changes.
- Maintaining project history.
- Supporting collaboration among developers.
- Restoring previous versions of files.
- Managing parallel development.
- Reducing the risk of source code loss.

Version Control Systems are broadly classified into:

- Local Version Control Systems
- Centralized Version Control Systems
- Distributed Version Control Systems

Git is a Distributed Version Control System.

6.2 Git

Git is an open-source distributed version control system used to track changes in source code during software development.

Git allows every developer to maintain a complete copy of the project repository on the local system.

Important features of Git include:

- Distributed development.
- Fast source code management.
- Branching and merging.
- Version tracking.
- Offline operations.
- Collaboration support.

The basic Git workflow is:

Working Directory

↓

Staging Area



Local Repository



Remote Repository

6.3 GitHub

GitHub is a cloud-based platform used to host and manage Git repositories.

It provides facilities for:

- Remote repository hosting.
- Source code collaboration.
- Branch management.
- Pull requests.
- Code review.
- Issue tracking.
- Project management.
- Continuous Integration and Continuous Deployment integration.

Git and GitHub are different technologies.

Git is a version control system used to manage source code versions.

GitHub is a cloud-based platform used to host Git repositories and support collaboration.

6.4 Git Repository

A Git repository is a directory that contains project files and maintains the complete history of changes made to those files.

A repository can be:

Local Repository: Stored on the developer's computer.

Remote Repository: Stored on a remote server such as GitHub.

The .git directory stores Git repository information and version history.

6.5 Git Clone

The git clone command creates a local copy of an existing remote repository.

Syntax:

`git clone <repository-url>`

Example:

`git clone <GitHub-repository-URL>`

The clone operation downloads:

- Project files.
- Repository history.
- Branch information.
- Git configuration.

6.6 Git Status

The git status command displays the current state of the working directory and staging area.

Command:

`git status`

It shows:

- Modified files.
- Untracked files.
- Staged files.
- Current branch.

6.7 Git Add

The git add command moves changes from the working directory to the staging area.

Syntax:

`git add <filename>`

To stage all modified files:

`git add .`

The staging area contains changes that will be included in the next commit.

6.8 Git Commit

A commit is a snapshot of the project at a particular point in time.

The git commit command stores staged changes in the local Git repository.

Syntax:

```
git commit -m "Commit message"
```

Example:

```
git commit -m "Add Student Task Manager application"
```

A meaningful commit message should clearly describe the changes made to the project.

6.9 Git Push

The git push command uploads commits from the local repository to the remote repository.

Syntax:

```
git push origin main
```

Where:

- origin represents the remote repository.
- main represents the branch being pushed.

The Git push operation updates the GitHub repository with local changes.

6.10 Git Pull

The git pull command downloads the latest changes from the remote repository and integrates them into the local repository.

Syntax:

```
git pull origin main
```

Git pull performs two operations:

Fetch

Merge

It helps developers synchronize the local repository with the remote repository.

6.11 Git Branch

A branch represents an independent line of development.

The default branch is generally named main.

A new branch can be created using:

```
git branch feature-ui
```

To switch to the branch:

```
git switch feature-ui
```

A branch can also be created and switched to using:

```
git switch -c feature-ui
```

Branching allows developers to develop new features without directly modifying the main branch.

Example:

main

|

|----- feature-ui

|

|----- feature-task

6.12 Git Merge

Git merge combines changes from one branch into another branch.

For example, to merge the feature-ui branch into the main branch:

Switch to the main branch:

```
git switch main
```

Merge the feature branch:

```
git merge feature-ui
```

The changes developed in the feature branch are integrated into the main branch.

6.13 Basic Git Workflow

The basic Git workflow for the Student Task Manager application is:

Create GitHub Repository

↓

Clone Repository

↓

Add Application Files

↓

Stage Changes

↓

Commit Changes

↓
Push to GitHub
↓
Create Feature Branch
↓
Modify Application
↓
Commit Changes
↓
Merge Feature Branch
↓
Push Updated Code
↓
Pull Latest Changes

7. Algorithm

Algorithm: Version Control of Student Task Manager Web Application Using Git

1. Start.
2. Log in to GitHub.
3. Create a new GitHub repository named devops-task-manager.
4. Copy the repository URL.
5. Open the terminal or Git Bash.
6. Clone the GitHub repository to the local system.
7. Navigate to the cloned repository directory.
8. Create the Student Task Manager Web Application files.
9. Check the repository status using git status.
10. Add application files to the staging area.
11. Commit the staged files with an appropriate commit message.
12. Push the committed files to the remote GitHub repository.

13. Create a new branch named feature-ui.
14. Switch to the feature-ui branch.
15. Modify the user interface of the web application.
16. Stage the modified files.
17. Commit the modifications.
18. Switch to the main branch.
19. Merge the feature-ui branch into the main branch.
20. Push the merged changes to the GitHub repository.
21. Pull the latest changes from the remote repository.
22. Verify the commit and branch history.
23. Stop.

8. Sample Input

Repository Name

devops-task-manager

Sample Application Files

devops-task-manager

|

|— index.html

|— style.css

|— script.js

Git Commands

git clone <GitHub-repository-URL>

cd devops-task-manager

git status

git add .

git commit -m "Add Student Task Manager application"

git push origin main

git switch -c feature-ui

git add .

git commit -m "Improve Task Manager user interface"

git switch main

git merge feature-ui

git push origin main

git pull origin main

9. Sample Output

Repository successfully cloned.

Student Task Manager application files added.

Changes staged successfully.

[main 8a7c21d] Add Student Task Manager application

3 files changed.

Application successfully pushed to GitHub.

Switched to a new branch 'feature-ui'.

User interface modifications completed.

[feature-ui 4b8e32a] Improve Task Manager user interface

Switched to branch 'main'.

Updating 8a7c21d..4b8e32a

Fast-forward

Branch 'feature-ui' successfully merged.

Changes successfully pushed to GitHub.

Already up to date.

Expected Git Repository Structure

GitHub Repository

```
|
|
├── devops-task-manager
|
└── index.html
```

└─ style.css

└─ script.js

Expected Branch Structure

main

|

|---- Initial Application Commit

|

|---- feature-ui

|

└─ UI Modification Commit

|

↓

Merge to main

10. Outcome

After completing this experiment, students will be able to:

- Create and manage Git repositories.
- Use Git for source code version control.
- Clone a remote GitHub repository.
- Stage and commit source code changes.
- Push local commits to a remote repository.
- Pull updated source code from a remote repository.
- Create and manage Git branches.
- Merge feature branches with the main branch.
- Apply Git version control practices to a web application project.

11. Viva Questions

1. What is a Version Control System?
2. What is Git?
3. Why is Git called a Distributed Version Control System?

4. What is GitHub?
5. Differentiate between Git and GitHub.
6. What is a Git repository?
7. Differentiate between a local repository and a remote repository.
8. What is the purpose of the git clone command?
9. What is the purpose of the git status command?
10. What is the staging area in Git?
11. What is the purpose of the git add command?
12. What is a Git commit?
13. Why should meaningful commit messages be used?
14. What is the purpose of the git push command?
15. What is the purpose of the git pull command?
16. Differentiate between git push and git pull.
17. What is a branch in Git?
18. Why are branches used in software development?
19. What is the purpose of the git switch command?
20. What is Git merge?
21. What is the main branch in Git?
22. What is the purpose of the origin keyword in Git?
23. What is the difference between the working directory and staging area?
24. What happens when git add . is executed?
25. What is the difference between git fetch and git pull?
26. What is a merge conflict?
27. When can a merge conflict occur?
28. What is the .git directory?
29. How can you view the commit history in Git?
30. What is the purpose of the git log command?

Assignment No: 2

Assignment No: 2

Title

Git Branching and Collaboration for Distributed Software Development

Course Outcome (CO) Mapped: CO1

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 02

1. Aim

To implement Git branching strategies using feature branches, pull requests, code review, and merging to simulate collaborative software development in a distributed team.

2. Problem Statement

Extend the **Student Task Manager Web Application** developed in Assignment No. 1 by applying Git branching and collaborative development practices.

The application development should be performed collaboratively using Git and GitHub.

Perform the following tasks:

1. Use the existing GitHub repository of the Student Task Manager Web Application.
2. Add a collaborator to the GitHub repository.
3. Clone the shared GitHub repository on the collaborator's system.
4. Create separate feature branches for application development.
5. Implement new features in independent branches.
6. Commit and push feature branch changes to GitHub.
7. Create a Pull Request for the implemented feature.
8. Review the changes made by another team member.
9. Approve and merge the Pull Request into the main branch.
10. Pull the latest changes from the main branch to synchronize the local repository.

The collaborative development workflow should simulate software development in a distributed development team.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Apply Git branching strategies for parallel software development.
- Create and manage feature branches.
- Collaborate using shared GitHub repositories.
- Push local branches to a remote repository.
- Create and manage Pull Requests.

- Perform basic code review using GitHub.
- Merge feature branches into the main branch.
- Synchronize local repositories with remote repositories.
- Understand collaborative development workflows used in distributed software teams.

4. Prerequisites

Students should be familiar with:

- Basic HTML, CSS, and JavaScript.
- Basic command-line operations.
- Git repository creation.
- Basic Git commands.
- Git clone, add, commit, push, and pull operations.
- Basic Git branching and merging.
- GitHub account and repository management.

5. Software and Hardware Requirements

Software

- Git
- GitHub Account
- Visual Studio Code or any suitable code editor
- Google Chrome / Microsoft Edge / any modern web browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 4 GB RAM
- Internet connectivity
- Minimum 1 GB free disk space

6. Theory

6.1 Collaborative Software Development

Collaborative software development is a development approach in which multiple developers work together on the same software project.

In large and distributed software teams, developers may work from different geographical locations and contribute to different application features.

A collaborative development environment requires mechanisms for:

- Sharing source code.
- Managing multiple developers.
- Tracking individual changes.

- Developing features independently.
- Reviewing source code.
- Integrating changes.
- Resolving conflicts.

Git and GitHub provide tools to support collaborative software development.

A typical collaborative workflow is:

Developer

↓

Clone Repository

↓

Create Feature Branch

↓

Develop Feature

↓

Commit Changes

↓

Push Feature Branch

↓

Create Pull Request

↓

Code Review

↓

Merge into Main Branch

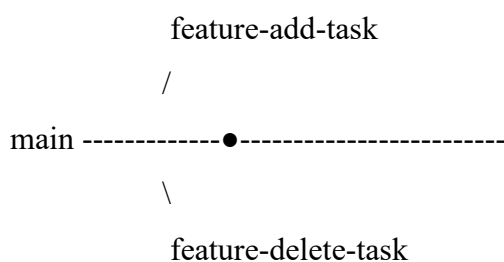
6.2 Git Branching

A branch in Git represents an independent line of development.

Branches allow developers to modify source code without directly affecting the stable version of the application.

The main branch generally contains the stable version of the project.

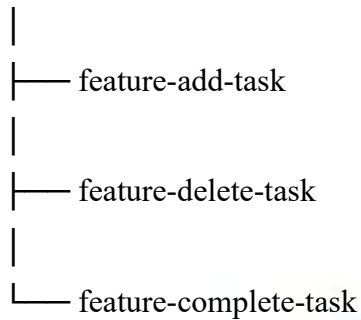
Example:



Different developers can work on different branches simultaneously.

For the Student Task Manager application:

main



Each feature is developed independently.

6.3 Feature Branching Strategy

Feature branching is a Git branching strategy in which a separate branch is created for every new feature or application modification.

Examples:

feature-add-task

feature-delete-task

feature-complete-task

feature-ui

The feature branch is created from the main branch.

Command:

git switch -c feature-add-task

The general feature branch workflow is:

main



Create Feature Branch



Develop Feature



Commit Changes



Push Branch



Pull Request



Code Review



Merge



main

Benefits of feature branches include:

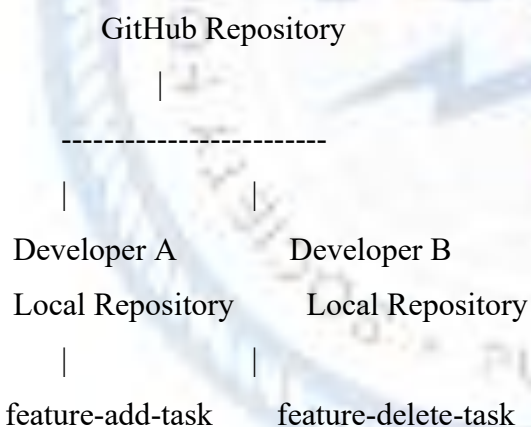
- Independent feature development.
- Protection of the main branch.
- Parallel development.
- Easier code review.
- Better source code organization.
- Simplified change tracking.

6.4 Distributed Development

Distributed development refers to a software development environment in which developers work on the same project from different systems or locations.

Each developer maintains a local copy of the Git repository.

Example:



Developers synchronize their changes through the remote GitHub repository.

Git supports distributed development because every developer maintains:

- Project files.
- Branch information.
- Commit history.
- Local repository.

6.5 GitHub Collaborator

A collaborator is a GitHub user who is provided access to contribute to a repository.

The repository owner can add collaborators to a repository.

The basic process is:

GitHub Repository



Settings



Collaborators



Add Collaborator



Send Invitation



Collaborator Accepts Invitation

After accepting the invitation, the collaborator can contribute to the project according to the assigned repository permissions.

6.6 Creating a Feature Branch

Before creating a feature branch, the developer should ensure that the main branch contains the latest source code.

`git switch main`

Pull the latest changes:

`git pull origin main`

Create a feature branch:

`git switch -c feature-add-task`

Verify the current branch:

`git branch`

Sample output:

* feature-add-task

main

The * symbol indicates the currently active branch.

6.7 Committing Feature Changes

After implementing a feature, the modified files are added to the staging area.

`git add .`

Commit the changes:

`git commit -m "Add task creation feature"`

A commit creates a snapshot of the feature implementation in the local repository.

6.8 Pushing a Feature Branch

A local feature branch must be pushed to GitHub before creating a Pull Request.

Command:

```
git push -u origin feature-add-task
```

The -u option establishes a tracking relationship between the local branch and the remote branch.

After setting the upstream branch, future changes can be pushed using:

```
git push
```

6.9 Pull Request

A Pull Request is a GitHub feature used to propose changes from one branch to another branch.

For example:

feature-add-task



Pull Request



main

A Pull Request allows team members to:

- View source code changes.
- Compare branches.
- Discuss implementation.
- Review code.
- Suggest modifications.
- Approve changes.
- Merge changes.

The Pull Request workflow is:

Feature Branch



Push to GitHub



Create Pull Request



Review Changes



Approve Changes



Merge Pull Request

6.10 Code Review

Code review is the process of examining source code changes before integrating them into the main project.

The objectives of code review include:

- Identifying programming errors.
- Improving code quality.
- Maintaining coding standards.
- Sharing knowledge among developers.
- Detecting unnecessary changes.
- Ensuring feature correctness.

During a Pull Request review, a reviewer may:

- Comment on source code.
- Suggest changes.
- Approve the Pull Request.
- Request modifications.

Example collaborative workflow:

Developer A



Develop Feature



Create Pull Request



Developer B



Review Code



Approve / Request Changes

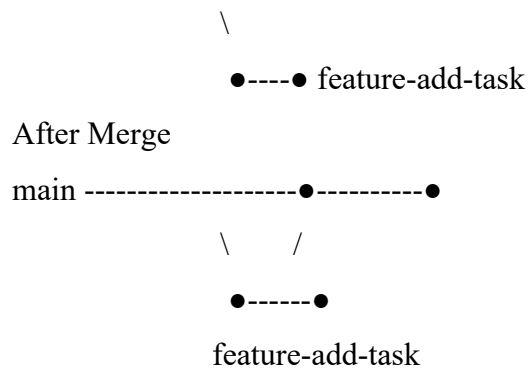
6.11 Merging a Pull Request

After a Pull Request is reviewed and approved, the changes can be merged into the main branch.

The merge operation integrates feature branch changes into the main branch.

Before Merge

main -----●



GitHub provides different merge options:

- Create a merge commit.
- Squash and merge.
- Rebasing and merge.

For this experiment, students may use **Create a merge commit**.

6.12 Repository Synchronization

After the Pull Request is merged, other developers must update their local repositories.

Switch to the main branch:

`git switch main`

Pull the latest changes:

`git pull origin main`

The local repository is synchronized with the remote GitHub repository.

The synchronization process is:

GitHub Main Branch



`git pull`



Local Main Branch

6.13 Collaborative Git Workflow

The complete collaborative Git workflow for the Student Task Manager application is:

GitHub Repository



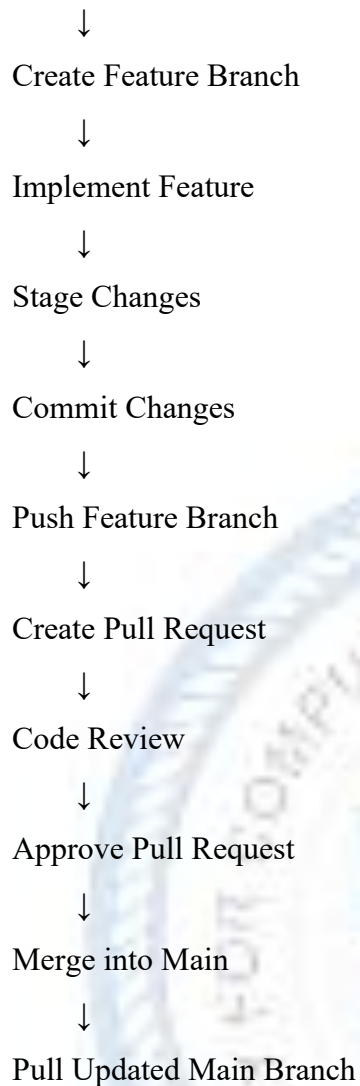
Add Collaborator



Clone Repository



Pull Latest Main Branch



7. Algorithm

Algorithm: Collaborative Development Using Git Branching and Pull Requests

1. Start.
2. Log in to GitHub.
3. Open the Student Task Manager GitHub repository.
4. Add another student as a repository collaborator.
5. The collaborator accepts the repository invitation.
6. Clone the shared repository to the collaborator's local system.
7. Navigate to the repository directory.
8. Switch to the main branch.
9. Pull the latest source code from the remote repository.
10. Create a new feature branch named feature-add-task.
11. Switch to the feature branch.
12. Implement the Add Task feature in the Student Task Manager application.

13. Check the modified files using git status.
14. Add the modified files to the staging area.
15. Commit the feature changes.
16. Push the feature branch to GitHub.
17. Create a Pull Request from feature-add-task to main.
18. Assign another team member as the reviewer.
19. Review the source code changes.
20. Approve the Pull Request.
21. Merge the Pull Request into the main branch.
22. Switch to the local main branch.
23. Pull the latest changes from the remote main branch.
24. Verify the integrated feature.
25. Stop.

8. Sample Input

GitHub Repository

devops-task-manager

Team Members

Student A : Repository Owner / Developer

Student B : Collaborator / Reviewer

Feature Branch

feature-add-task

Git Commands

git clone <GitHub-repository-URL>

cd devops-task-manager

git switch main

git pull origin main

git switch -c feature-add-task

git status

git add .

git commit -m "Add task creation feature"

git push -u origin feature-add-task

Pull Request Configuration

Base Branch : main

Compare Branch : feature-add-task

Pull Request Title:

Add Task Creation Feature

Reviewer:

Student B

After the Pull Request is merged:

git switch main

git pull origin main

9. Sample Output

Repository cloned successfully.

Already on 'main'.

Main branch synchronized with remote repository.

Switched to a new branch 'feature-add-task'.

Task creation feature implemented.

Changes staged successfully.

[feature-add-task 7d3a421] Add task creation feature

Feature branch pushed successfully.

Pull Request created:

feature-add-task → main

Code review completed.

Pull Request approved.

Pull Request merged successfully.

Switched to branch 'main'.

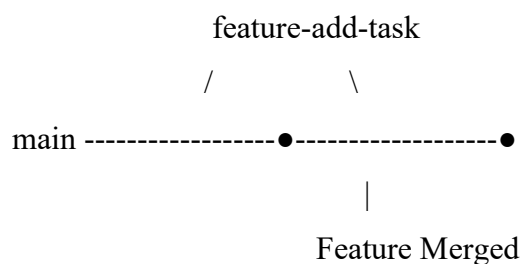
Updating 5c7b123..7d3a421

Fast-forward.

Latest changes successfully pulled from the remote repository.

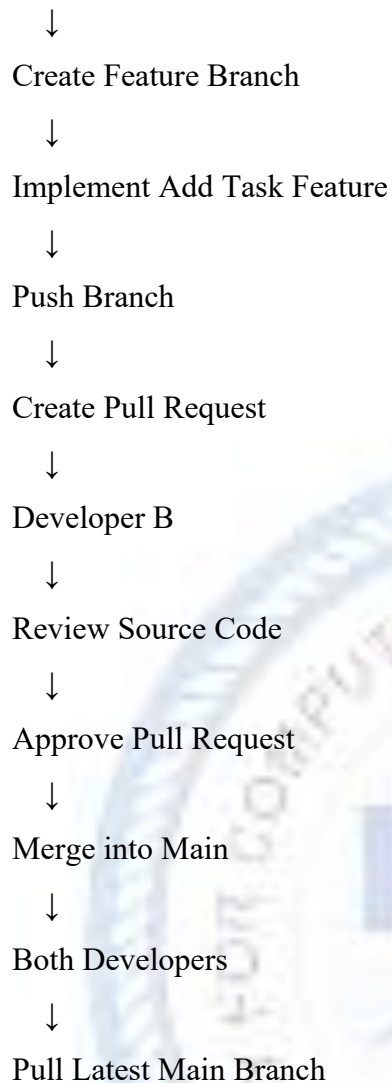
Add Task feature successfully integrated into the Student Task Manager application.

Expected Branch Structure



Expected Collaborative Workflow

Developer A



10. Outcome

After completing this experiment, students will be able to:

- Apply Git branching strategies in software development.
- Create and manage feature branches.
- Collaborate using shared GitHub repositories.
- Implement application features independently.
- Push local branches to remote repositories.
- Create Pull Requests using GitHub.
- Perform basic source code review.
- Approve and merge Pull Requests.
- Synchronize local and remote Git repositories.
- Apply collaborative development practices used in distributed software teams.

11. Viva Questions

1. What is collaborative software development?

2. What is distributed software development?
3. Why is Git suitable for distributed development?
4. What is a branch in Git?
5. Why are branches used in software development?
6. What is a feature branch?
7. Explain the feature branching strategy.
8. What is the purpose of the main branch?
9. How do you create a new branch in Git?
10. What is the purpose of the git switch command?
11. Differentiate between git branch and git switch.
12. What does git switch -c do?
13. What is a GitHub collaborator?
14. How can a collaborator be added to a GitHub repository?
15. What is a remote branch?
16. Differentiate between a local branch and a remote branch.
17. What is the purpose of git push -u origin branch-name?
18. What is an upstream branch?
19. What is a Pull Request?
20. Why are Pull Requests used?
21. What is code review?
22. What are the benefits of code review?
23. Who is a reviewer in a Pull Request?
24. What actions can a Pull Request reviewer perform?
25. What is Git merge?
26. What happens when a Pull Request is merged?
27. What is a merge commit?
28. What is Squash and Merge?
29. What is Rebase and Merge?
30. Differentiate between merge and rebase.
31. What is a merge conflict?
32. Why do merge conflicts occur?
33. How can merge conflicts be resolved?
34. Why should developers pull the latest main branch before creating a feature branch?
35. Why should developers avoid directly modifying the main branch in collaborative

projects?



Assignment No: 3

Assignment No: 3

Title

Build Automation of a Web Application Using Jenkins

Course Outcome (CO) Mapped: CO1

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 04

1. Aim

To configure Jenkins to retrieve the Student Task Manager Web Application from GitHub and execute the application build process successfully.

2. Problem Statement

Configure Jenkins for build automation of the Student Task Manager Web Application stored in a GitHub repository.

The application should be configured as a Jenkins Freestyle Project. Jenkins should retrieve the latest source code from GitHub, install the required dependencies, execute the build process, and display the build status.

The assignment should demonstrate:

1. Installation and configuration of Jenkins.
2. Integration of Jenkins with GitHub.
3. Creation of a Jenkins Freestyle Project.
4. Automatic retrieval of application source code.
5. Installation of Node.js dependencies.
6. Execution of the application build.
7. Analysis of Jenkins Console Output and build status.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand the role of Jenkins in DevOps.
- Explain build automation.
- Configure a Jenkins Freestyle Project.
- Integrate Jenkins with a GitHub repository.

- Execute application build commands using Jenkins.
- Analyze Jenkins build status and Console Output.
- Understand the difference between manual and automated build processes.

4. Prerequisites

Students should be familiar with:

- Basic Git and GitHub operations.
- Node.js and npm.
- Basic command-line operations.
- Student Task Manager Web Application used in previous assignments.

5. Software and Hardware Requirements

Software

- Jenkins
- Java JDK supported by Jenkins
- Git
- Node.js and npm
- GitHub Account
- Visual Studio Code
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- Internet connectivity
- Minimum 5 GB free disk space

6. Theory

6.1 Build Automation

Build automation is the process of automatically executing the steps required to prepare an application for execution or deployment.

A typical build process may include:

Source Code



Install Dependencies



Validate Application



Build Application



Build Result

Without build automation, developers manually execute build commands whenever the source code changes.

With build automation, a build tool or automation server executes the configured commands consistently.

Benefits of build automation include:

- Reduced manual effort.
- Consistent build execution.
- Faster feedback.
- Early detection of build errors.
- Improved development productivity.

6.2 Jenkins

Jenkins is an open-source automation server widely used for automating software build, testing, and deployment activities.

Jenkins supports:

- Source code integration.
- Build automation.
- Automated testing.
- CI/CD pipelines.
- Plugin-based integration.
- Build status monitoring.

The basic Jenkins build workflow is:

Developer



Push Code to GitHub



Jenkins



Retrieve Source Code



Install Dependencies



Execute Build



SUCCESS / FAILURE

6.3 Jenkins Freestyle Project

A Jenkins Freestyle Project is a simple Jenkins job in which build configuration is performed using the Jenkins graphical interface.

A Freestyle Project allows users to configure:

- Source Code Management.
- Git repository.
- Branch information.
- Build commands.
- Build triggers.
- Post-build actions.

In this assignment, a Freestyle Project is used to understand the basic Jenkins build process.

6.4 Jenkins and GitHub Integration

GitHub stores the application source code, while Jenkins retrieves the source code for build execution.

The workflow is:

GitHub Repository



Jenkins Git Configuration



Clone Repository



Jenkins Workspace



Execute Build

The GitHub repository URL is configured in the Source Code Management section of the Jenkins Job.

Jenkins uses Git to retrieve the application source code.

6.5 Jenkins Workspace

A Jenkins Workspace is a directory in which Jenkins stores the source code and executes build commands.

The process is:

GitHub



Source Code Retrieved



Jenkins Workspace



npm install



npm run build

Each Jenkins Job normally has its own Workspace.

6.6 Build Process for the Student Task Manager

The Student Task Manager is a Node.js-based web application.

The build process consists of:

Retrieve Source Code



npm install



npm run build



Build Result

The command:

npm install

installs the dependencies defined in package.json.

The command:

npm run build

executes the build script configured in package.json.

Example:

```
"scripts": {  
  "start": "node app.js",  
  "build": "node build.js"  
}
```

6.7 Build Script

Create a file named:

build.js

A simple build validation script is:

```
const fs = require('fs');  
  
const requiredFiles = [  
  'app.js',  
  'package.json',  
  'public/index.html'  
];  
  
console.log('Starting application build validation...');
```

```
for (const file of requiredFiles) {  
  if (!fs.existsSync(file)) {  
    console.error(`Build Failed: ${file} not found`);  
    process.exit(1);  
  }  
}  
  
console.log('Application files validated successfully.');
```

The script checks whether the required application files are available.

If all files exist:

Build Completed



Exit Code 0



Jenkins SUCCESS

If a required file is missing:

Build Failed



Exit Code 1



Jenkins FAILURE

6.8 Jenkins Build Status

Jenkins displays the result of every build.

Common build results are:

Build Status Meaning

SUCCESS Build completed successfully

FAILURE Build process failed

Build Status Meaning

UNSTABLE Build completed with some problems

ABORTED Build was manually stopped

The Jenkins Console Output provides detailed information about each build step.

Example:

Cloning Git Repository

npm install

npm run build

Application files validated successfully.

Build completed successfully.

Finished: SUCCESS

6.9 Offline and Online Jenkins Setup

Jenkins can be installed locally or hosted on a remote server.

Local Jenkins

Jenkins runs on the student's computer.

Student Computer



Local Jenkins



GitHub

Advantages:

- Suitable for laboratory practice.
- Easy to configure.
- No cloud cost.
- Simple troubleshooting.

Remote Jenkins

Jenkins runs on a cloud server or organization server.

Developer



GitHub



Remote Jenkins Server



Build Environment

Remote Jenkins is more suitable for team-based and enterprise environments.

For this laboratory assignment, local Jenkins is recommended.

6.10 Jenkins in Industry

In industry, Jenkins is commonly used as part of a CI/CD workflow.

Developer



Git Commit



GitHub / GitLab



Jenkins



Build



Automated Tests



Security Checks



Docker Image



Deployment

Modern industry environments generally use Pipeline as Code rather than only Freestyle Projects.

However, the Freestyle Project is used in this assignment to introduce students to Jenkins build configuration.

Jenkins Pipeline and automated testing are covered in Assignment No. 6.

7. Procedure

Step 1: Verify the Application

Open the Student Task Manager project.

Verify the following files:

devops-task-manager

```
|  
|  
|— public  
|  |  
|  |— index.html  
|  
|— app.js  
|— build.js  
|— package.json  
|— package-lock.json
```

Step 2: Configure the Build Script

Verify the package.json file.

```
"scripts": {  
  "start": "node app.js",  
  "build": "node build.js"  
}
```

Execute locally:

npm install

npm run build

Verify that the build completes successfully.

Step 3: Push the Application to GitHub

Execute:

git add .

git commit -m "Add application build configuration"

git push origin main

Verify the application files in the GitHub repository.

Step 4: Start Jenkins

Start Jenkins and open the Jenkins Dashboard in a web browser.

Complete the initial Jenkins configuration if required.

Verify that Git is available to Jenkins.

Step 5: Create a Jenkins Freestyle Project

From the Jenkins Dashboard:

New Item



Enter Job Name



Select Freestyle Project



Click OK

Job Name:

task-manager-build

Step 6: Configure GitHub Repository

Navigate to:

Source Code Management



Git

Enter the GitHub Repository URL.

Configure the branch as:

***/main**

Add GitHub credentials if the repository is private.

Step 7: Configure Build Steps

Navigate to:

Build Steps



Add Build Step

For Windows, select Execute Windows Batch Command and enter:

npm install

npm run build

For Linux, select Execute Shell and enter:

npm install

npm run build

Step 8: Save and Execute the Build

Click:

Save



Build Now

Jenkins retrieves the application from GitHub and executes the build commands.

Step 9: Verify Console Output

Open:

Build Number



Console Output

Verify:

Application files validated successfully.

Build completed successfully.

Finished: SUCCESS

Step 10: Observe Build Failure

Temporarily rename or remove a required application file.

For example:

`public/index.html`

Commit and push the change.

Execute the Jenkins build again.

Observe:

Build Failed: `public/index.html` not found

Finished: FAILURE

Restore the file after completing the experiment.

8. Algorithm

Algorithm: Jenkins Build Automation

1. Start.
2. Open the Student Task Manager application.
3. Configure the application build script.
4. Execute the build locally.
5. Push the application source code to GitHub.
6. Start Jenkins.
7. Create a Jenkins Freestyle Project.
8. Configure Git as Source Code Management.
9. Enter the GitHub repository URL.
10. Configure the main branch.
11. Add the dependency installation command.
12. Add the application build command.
13. Save the Jenkins Job.
14. Execute the Jenkins build.
15. Jenkins retrieves the source code from GitHub.
16. Jenkins installs application dependencies.
17. Jenkins executes the build script.
18. Verify the Console Output.

19. Verify the build status.

20. Stop.

9. Sample Input

Jenkins Job Name

task-manager-build

Repository Branch

*/main

Build Commands

npm install

npm run build

10. Sample Output

Successful Build

Starting application build validation...

Application files validated successfully.

Build completed successfully.

Finished: SUCCESS

Failed Build

Starting application build validation...

Build Failed: public/index.html not found

Build step marked build as failure.

Finished: FAILURE

11. Outcome

After completing this experiment, students will be able to:

- Configure Jenkins for build automation.
- Create a Jenkins Freestyle Project.
- Integrate Jenkins with a GitHub repository.
- Retrieve application source code using Jenkins.
- Execute Node.js build commands using Jenkins.

- Analyze successful and failed Jenkins builds.
- Understand the role of build automation in DevOps.

12. Viva Questions

1. What is build automation?
2. What is Jenkins?
3. What is the role of Jenkins in DevOps?
4. What is a Jenkins Freestyle Project?
5. What is Source Code Management?
6. How does Jenkins retrieve source code from GitHub?
7. What is a Jenkins Workspace?
8. What is the purpose of npm install?
9. What is the purpose of npm run build?
10. What does exit code 0 indicate?
11. What does a non-zero exit code indicate?
12. What does Finished: SUCCESS indicate?
13. What is Jenkins Console Output?
14. Differentiate between local and remote Jenkins.
15. How is Jenkins used in an industry CI/CD workflow?

Assignment No: 4

Assignment No: 4

Title

Containerization of a Web Application Using Docker

Course Outcome (CO) Mapped: CO2

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 04

1. Aim

To containerize the Student Task Manager Web Application using Docker and execute the application consistently across different environments without manual dependency setup.

2. Problem Statement

Containerize the **Student Task Manager Web Application** using Docker.

Create a Dockerfile that defines the application runtime environment, dependencies, source code, application port, and execution command. Build a Docker Image and run the application as a Docker Container.

The assignment should demonstrate:

1. Installation and verification of Docker.
2. Creation of a Dockerfile.
3. Creation of a .dockerignore file.
4. Building a Docker Image.
5. Running the application as a Docker Container.
6. Configuring container port mapping.
7. Verifying the containerized application.
8. Managing Docker Images and Containers.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand containerization and Docker.
- Explain Docker Images and Containers.
- Create a Dockerfile.
- Build a Docker Image.
- Run and manage Docker Containers.

- Configure Docker port mapping.
- Use .dockerignore.
- Understand application portability using containers.

4. Prerequisites

Students should be familiar with:

- Basic Node.js and npm.
- Basic command-line operations.
- Student Task Manager Web Application.
- Basic Git and GitHub concepts.

5. Software and Hardware Requirements

Software

- Docker Desktop / Docker Engine
- Node.js and npm
- Visual Studio Code
- Git
- Web Browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- 64-bit processor
- Minimum 10 GB free disk space
- Internet connectivity for downloading Docker Images

6. Theory

6.1 Containerization

Containerization is a software deployment approach in which an application and its required dependencies are packaged into a standardized unit called a **Container**.

Without containerization:

Application

↓

Install Node.js



Install Dependencies



Configure Environment



Run Application

Different systems may have different runtime versions and configurations, which can lead to the common problem:

"It works on my machine."

With containerization:

Application

+

Dependencies

+

Runtime Environment



Container

The same containerized application can run consistently on any system supporting the container runtime.

6.2 Docker

Docker is a platform used to build, package, distribute, and run applications using containers.

The basic Docker workflow is:

Application Source Code



Dockerfile



docker build



Docker Image

↓

docker run

↓

Docker Container

Docker provides:

- Application isolation.
- Portability.
- Consistent runtime environments.
- Fast application deployment.
- Efficient resource utilization.
- Support for microservices and cloud-native applications.

6.3 Virtual Machines and Containers

Virtual Machines and Containers provide application isolation, but their architecture is different.

Virtual Machine	Container
Includes Guest Operating System	Shares Host OS Kernel
Requires more resources	Lightweight
Larger size	Smaller size
Slower startup	Faster startup
Hardware-level virtualization	OS-level virtualization

Virtual Machine architecture:

Hardware

↓

Host Operating System

↓

Hypervisor

↓

Guest OS

↓

Application

Container architecture:

Hardware



Host Operating System



Docker Engine



Container



Application

Containers are commonly used in DevOps because they provide fast and consistent application deployment.

6.4 Docker Image

A Docker Image is a read-only template containing the application, runtime, libraries, and configuration required to create a Container.

Example:

Node.js Runtime

+

Application Code

+

Dependencies

+

Configuration



Docker Image

In this experiment, the Docker Image is:

task-manager:1.0

Here:

task-manager → Image Name

1.0 → Image Tag

A single Docker Image can be used to create multiple Containers.

6.5 Docker Container

A Docker Container is a running instance of a Docker Image.

Docker Image

↓

docker run

↓

Docker Container

Example:

task-manager:1.0

↓

task-manager-container

The Container executes the Student Task Manager application in an isolated environment.

6.6 Dockerfile

A Dockerfile is a text file containing instructions used to build a Docker Image.

A Dockerfile defines:

- Base Image.
- Working directory.
- Application files.
- Dependencies.
- Application port.
- Startup command.

For the Student Task Manager application:

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]

6.7 Dockerfile Instructions

FROM

FROM node:20-alpine

Specifies the base Docker Image.

The Node.js Alpine Image provides a lightweight Node.js runtime environment.

WORKDIR

WORKDIR /app

Defines the working directory inside the Container.

Subsequent Dockerfile commands execute from this directory.

COPY

COPY package*.json ./

Copies the Node.js package files into the Docker Image.

The command:

COPY . .

copies the application source code.

RUN

RUN npm install

Executes a command while building the Docker Image.

The command installs the application dependencies.

EXPOSE

EXPOSE 3000

Documents that the application uses port 3000 inside the Container.

CMD

CMD ["npm", "start"]

Defines the default command executed when the Container starts.

6.8 .dockerignore

The .dockerignore file specifies files and directories that should not be copied into the Docker Image.

Example:

node_modules

.git

.gitignore

README.md

The benefits of .dockerignore include:

- Smaller Docker build context.
- Faster Image builds.
- Reduced Image size.
- Prevention of unnecessary files from entering the Image.

6.9 Docker Build Process

The command:

`docker build -t task-manager:1.0 .`

builds the Docker Image.

The workflow is:

Dockerfile



Read Instructions



Create Image Layers



Install Dependencies



Copy Application



Create Docker Image

The option:

`-t`

is used to assign a name and tag to the Image.

The dot (.) specifies the current directory as the Docker build context.

6.10 Docker Port Mapping

The application runs on port 3000 inside the Container.

The Container port is not directly accessible from the Host system unless it is mapped.

Example:

```
docker run -d -p 3000:3000 --name task-manager-container task-manager:1.0
```

The port mapping is:

Host Port : Container Port

3000 : 3000

The communication flow is:

Web Browser



localhost:3000



Host Port 3000



Container Port 3000



Student Task Manager

6.11 Docker Container Lifecycle

A Docker Container may have different lifecycle states.

Created



Running



Stopped



Removed

Common Docker commands are:

Command	Purpose
docker ps	Display running Containers
docker ps -a	Display all Containers
docker stop	Stop a Container
docker start	Start a stopped Container
docker rm	Remove a Container
docker logs	Display Container logs
docker images	Display Docker Images
docker rmi	Remove a Docker Image

6.12 Docker in DevOps and Industry

Docker is commonly used in DevOps pipelines to create consistent deployment artifacts.

A typical workflow is:

Developer



Git Repository



Jenkins CI Pipeline



Build Application



Create Docker Image



Container Registry



Kubernetes



Application Deployment

In industry, Docker Images are generally stored in a **Container Registry**, such as Docker Hub or a private cloud registry.

The same Docker Image is promoted across environments:

Development

↓

Testing

↓

Staging

↓

Production

This reduces environment-related configuration differences.

In this experiment, the Docker Image is created and executed locally. In Assignment No. 5, the same Dockerized application will be deployed using Kubernetes.

7. Procedure

Step 1: Verify Docker Installation

Execute:

```
docker --version
```

Verify that Docker is installed.

Execute:

```
docker info
```

Ensure that the Docker Engine is running.

Step 2: Open the Student Task Manager Project

Navigate to the project directory:

```
cd devops-task-manager
```

Verify the project structure:

```
devops-task-manager
```

```
|
```

```
| — public
```

```
|   └─ index.html
```

```
|
```

- |— app.js
- |— build.js
- |— package.json
- |— package-lock.json

Step 3: Verify Application Configuration

Ensure that app.js listens on:

```
const PORT = 3000;
```

The application should listen on all network interfaces:

```
app.listen(PORT, '0.0.0.0', () => {  
  console.log(`Student Task Manager running on port ${PORT}`);  
});
```

Using 0.0.0.0 allows the application to accept connections from outside the Container.

Step 4: Create .dockerignore

Create a file named:

.dockerignore

Add:

node_modules

.git

.gitignore

README.md

Step 5: Create Dockerfile

Create a file named:

Dockerfile

Add:

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]

Save the Dockerfile.

Step 6: Build the Docker Image

Execute:

```
docker build -t task-manager:1.0 .
```

Wait until the Image build completes successfully.

Step 7: Verify the Docker Image

Execute:

```
docker images
```

Verify:

```
task-manager 1.0
```

Step 8: Run the Docker Container

Execute:

```
docker run -d -p 3000:3000 --name task-manager-container task-manager:1.0
```

The options used are:

-d → Run Container in detached mode

-p → Configure port mapping

--name → Assign Container name

Step 9: Verify the Running Container

Execute:

```
docker ps
```

Verify that:

```
task-manager-container
```

is running.

Step 10: Access the Application

Open the web browser.

Access:

```
http://localhost:3000
```

Verify that the Student Task Manager application is displayed.

Step 11: View Container Logs

Execute:

```
docker logs task-manager-container
```

Expected output:

Student Task Manager running on port 3000

Step 12: Stop and Start the Container

Stop the Container:

```
docker stop task-manager-container
```

Verify:

```
docker ps
```

Start the Container again:

```
docker start task-manager-container
```

Verify the application in the browser.

Step 13: Remove the Container

Stop the Container:

```
docker stop task-manager-container
```

Remove the Container:

```
docker rm task-manager-container
```

Verify:

```
docker ps -a
```

8. Algorithm

Algorithm: Dockerization of Student Task Manager

1. Start.
2. Verify Docker installation.
3. Open the Student Task Manager project.
4. Verify the application port configuration.
5. Configure the application to listen on 0.0.0.0.
6. Create the .dockerignore file.

7. Create the Dockerfile.
8. Select the Node.js base Image.
9. Configure the working directory.
10. Copy the package files.
11. Install application dependencies.
12. Copy the application source code.
13. Expose application port 3000.
14. Configure the application startup command.
15. Build the Docker Image.
16. Verify the Docker Image.
17. Create and start the Docker Container.
18. Configure Host-to-Container port mapping.
19. Verify the running Container.
20. Access the Student Task Manager application.
21. Verify the Container logs.
22. Stop and start the Container.
23. Remove the Container.
24. Stop.

9. Sample Input

Docker Image Name

task-manager

Docker Image Tag

1.0

Container Name

task-manager-container

Application Port

3000

Build Command

docker build -t task-manager:1.0 .

Container Run Command

```
docker run -d -p 3000:3000 --name task-manager-container task-manager:1.0
```

10. Sample Output

Docker Image Build

Successfully built Docker Image

task-manager:1.0

Docker Images

REPOSITORY	TAG
task-manager	1.0

Running Container

CONTAINER NAME
task-manager-container

STATUS
Up

Container Logs

Student Task Manager running on port 3000

Application Output

Student Task Manager

Application successfully running inside Docker Container.

11. Outcome

After completing this experiment, students will be able to:

- Explain containerization and Docker.
- Differentiate between Docker Images and Containers.
- Create a Dockerfile.
- Use .dockerignore.
- Build a Docker Image.
- Run an application inside a Docker Container.
- Configure Docker port mapping.
- Manage Docker Containers.

- Understand the use of Docker in DevOps application deployment.

12. Viva Questions

1. What is containerization?
2. What is Docker?
3. What is a Docker Image?
4. What is a Docker Container?
5. Differentiate between a Docker Image and a Container.
6. Differentiate between a Virtual Machine and a Container.
7. What is a Dockerfile?
8. What is the purpose of the FROM instruction?
9. What is WORKDIR?
10. What is the difference between RUN and CMD?
11. What is the purpose of EXPOSE?
12. What is .dockerignore?
13. What is Docker port mapping?
14. What does 3000:3000 represent?
15. Why is 0.0.0.0 used in the Node.js application?
16. What is detached mode in Docker?
17. What is the purpose of docker ps?
18. How can Container logs be viewed?
19. What is a Container Registry?
20. How is Docker used in a DevOps CI/CD workflow?

Assignment No: 5

Assignment No: 5

Title

Deployment and Scaling of a Containerized Web Application Using Kubernetes

Course Outcome (CO) Mapped: CO2

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 04

1. Aim

To deploy the Dockerized Student Task Manager Web Application on Kubernetes using Minikube, configure application scaling, and make the application accessible to users.

2. Problem Statement

Deploy the **Dockerized Student Task Manager Web Application** developed in Assignment No. 4 on a local Kubernetes cluster using Minikube.

Create Kubernetes Deployment and Service configurations to manage application Pods and provide access to the application. Scale the application to multiple replicas and verify Kubernetes Pod management.

The assignment should demonstrate:

1. Creation of a local Kubernetes cluster using Minikube.
2. Deployment of a Dockerized application.
3. Creation and management of Kubernetes Pods.
4. Creation of a Kubernetes Deployment.
5. Application scaling using replicas.
6. Creation of a Kubernetes Service.
7. Accessing the deployed application.
8. Verification of Kubernetes self-healing.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand Kubernetes and container orchestration.
- Explain Kubernetes cluster architecture.
- Understand Pods, Deployments, and Services.
- Deploy a Dockerized application on Kubernetes.
- Scale an application using replicas.
- Expose a Kubernetes application using a Service.
- Use Minikube and kubectl commands.

- Understand Kubernetes self-healing.

4. Prerequisites

Students should be familiar with:

- Docker Images and Containers.
- Dockerfile.
- Basic command-line operations.
- YAML basics.
- Student Task Manager Docker Image created in Assignment No. 4.

5. Software and Hardware Requirements

Software

- Docker Desktop / Docker Engine
- Minikube
- kubectl
- Visual Studio Code
- Web Browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- 64-bit processor
- Hardware virtualization support
- Minimum 10 GB free disk space
- Internet connectivity for initial installation

6. Theory

6.1 Container Orchestration

Docker allows applications to run inside Containers. However, managing a large number of Containers manually becomes difficult.

Consider:

Application

↓

Container 1

Container 2

Container 3

Container 4

Container 5

Administrators must manage:

- Container deployment.
- Application scaling.
- Container failures.
- Network communication.
- Application availability.

Container orchestration automates the deployment, scaling, networking, and management of Containers.

6.2 Kubernetes

Kubernetes is an open-source container orchestration platform used to deploy, manage, and scale containerized applications.

The basic workflow is:

Dockerized Application



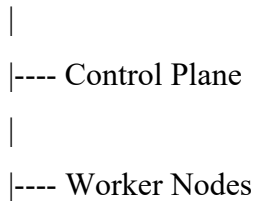
Kubernetes provides:

- Automated deployment.
- Application scaling.
- Self-healing.
- Service discovery.
- Load distribution.
- Rolling updates.
- Container lifecycle management.

6.3 Kubernetes Cluster Architecture

A Kubernetes Cluster consists of:

Kubernetes Cluster



Control Plane

The Control Plane manages the Kubernetes Cluster.

Important components include:

- API Server.
- Scheduler.
- Controller Manager.
- etcd.

Worker Node

A Worker Node runs application workloads.

Important components include:

- kubelet.
- Container Runtime.
- kube-proxy.
- Application Pods.

The simplified architecture is:

Control Plane



Manage Cluster



Worker Node



Application Pods

6.4 Minikube

Minikube is a tool used to create a local Kubernetes Cluster.

It is suitable for:

- Learning Kubernetes.
- Laboratory experiments.
- Local application testing.

- Kubernetes development.

The command:

`minikube start`

creates and starts a local Kubernetes Cluster.

The cluster status can be verified using:

`minikube status`

For this laboratory assignment, Minikube is recommended because students can perform Kubernetes operations locally without cloud infrastructure.

6.5 kubectl

kubectl is the command-line tool used to communicate with a Kubernetes Cluster.

The workflow is:

User



kubectl



Kubernetes API Server



Kubernetes Cluster

Common commands are:

Command	Purpose
<code>kubectl get pods</code>	Display Pods
<code>kubectl get deployments</code>	Display Deployments
<code>kubectl get svc</code>	Display Services
<code>kubectl apply -f</code>	Create or update resources
<code>kubectl describe</code>	Display resource details
<code>kubectl logs</code>	Display Pod logs
<code>kubectl delete</code>	Delete Kubernetes resources
<code>kubectl scale</code>	Scale a Deployment

6.6 Kubernetes Pod

A Pod is the smallest deployable unit in Kubernetes.

A Pod contains one or more Containers.

Pod



Container



Student Task Manager

In this assignment:

Student Task Manager Pod



task-manager Container

Pods are managed by Kubernetes.

A Pod may be deleted and recreated with a different name or IP address.

Therefore, applications should not depend directly on a specific Pod.

6.7 Kubernetes Deployment

A Deployment is a Kubernetes resource used to manage application Pods.

A Deployment defines the **desired state** of an application.

Example:

Desired Replicas = 3

Kubernetes ensures:

Pod 1 → Running

Pod 2 → Running

Pod 3 → Running

If one Pod fails:

Pod 2 → Failed



Kubernetes Detects Failure



New Pod Created

This behaviour is known as **self-healing**.

6.8 Kubernetes Deployment YAML

Kubernetes resources are commonly defined using YAML configuration files.

Create:

deployment.yaml

Example:

apiVersion: apps/v1

kind: Deployment

metadata:

name: task-manager-deployment

spec:

replicas: 2

selector:

matchLabels:

app: task-manager

template:

metadata:

labels:

app: task-manager

spec:

containers:

- name: task-manager-container

image: task-manager:1.0

imagePullPolicy: Never

ports:

- containerPort: 3000

The configuration specifies:

- Deployment name.
- Number of replicas.
- Pod labels.
- Docker Image.
- Container port.

6.9 Replicas and Scaling

A Replica represents a copy of an application Pod.

Example:

replicas: 2

Kubernetes creates:

Task Manager Pod 1

Task Manager Pod 2

The application can be scaled using:

kubectl scale deployment task-manager-deployment --replicas=4

The desired state becomes:

Desired Pods = 4

Kubernetes creates additional Pods automatically.

Scaling helps applications handle increased workload and improves availability.

6.10 Kubernetes Service

Pods are temporary and their IP addresses may change.

A Kubernetes Service provides a stable network endpoint for accessing a group of Pods.

User



Kubernetes Service



Pod 1

Pod 2

Pod 3

The Service identifies Pods using labels.

In this experiment, a **NodePort Service** is used.

6.11 NodePort Service

A NodePort Service exposes an application outside the Kubernetes Cluster.

Create:

service.yaml

Example:

apiVersion: v1

kind: Service

metadata:

name: task-manager-service

spec:

type: NodePort

selector:

app: task-manager

ports:

- protocol: TCP

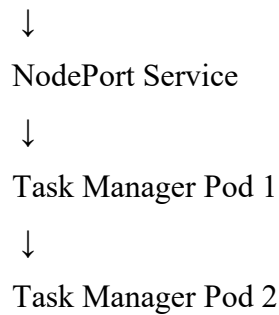
port: 3000

targetPort: 3000

The Service routes user requests to the application Pods.

The architecture becomes:

User



6.12 Kubernetes Self-Healing

Kubernetes continuously compares the actual application state with the desired state.

Example:

Desired Pods = 2

Actual Pods = 2

If a Pod is deleted:

Desired Pods = 2

Actual Pods = 1

Kubernetes automatically creates a replacement Pod.

Actual Pods = 2

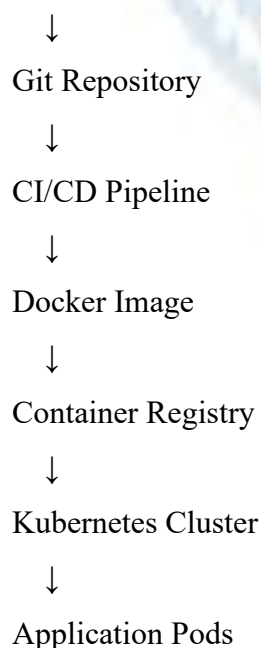
This capability is known as **self-healing**.

6.13 Kubernetes in Industry

In industry, Kubernetes is commonly used to deploy microservices and cloud-native applications.

A typical deployment workflow is:

Developer





Services



Users

Production environments generally use managed Kubernetes platforms such as:

- Amazon EKS.
- Azure Kubernetes Service.
- Google Kubernetes Engine.

For laboratory practice, Minikube provides a local Kubernetes environment.

7. Procedure

Step 1: Verify the Docker Image

Execute:

`docker images`

Verify:

`task-manager 1.0`

Step 2: Start Minikube

Execute:

`minikube start`

Verify the cluster:

`minikube status`

Step 3: Verify Kubernetes Cluster

Execute:

`kubectl cluster-info`

Verify the Kubernetes Node:

`kubectl get nodes`

The Node should display:

Ready

Step 4: Load the Docker Image into Minikube

Execute:

`minikube image load task-manager:1.0`

Verify:

`minikube image ls`

The Docker Image should be available inside Minikube.

Step 5: Create Kubernetes Deployment

Create a file named:

deployment.yaml

Add:

apiVersion: apps/v1

kind: Deployment

metadata:

name: task-manager-deployment

spec:

replicas: 2

selector:

matchLabels:

app: task-manager

template:

metadata:

labels:

app: task-manager

spec:

containers:

- name: task-manager-container

image: task-manager:1.0

imagePullPolicy: Never

ports:

- containerPort: 3000

Step 6: Deploy the Application

Execute:

kubectl apply -f deployment.yaml

Expected output:

deployment.apps/task-manager-deployment created

Step 7: Verify the Deployment and Pods

Execute:

kubectl get deployments

Execute:

kubectl get pods

Expected:

NAME	READY	STATUS
task-manager-deployment-xxxxx-aaaaa	1/1	Running
task-manager-deployment-xxxxx-bbbbb	1/1	Running

Step 8: Create Kubernetes Service

Create:

service.yaml

Add:

apiVersion: v1

kind: Service

metadata:

name: task-manager-service

spec:

type: NodePort

selector:

app: task-manager

ports:

- protocol: TCP

port: 3000

targetPort: 3000

Step 9: Deploy the Service

Execute:

kubectl apply -f service.yaml

Verify:

kubectl get svc

Step 10: Access the Application

Execute:

minikube service task-manager-service

The Student Task Manager application opens in the browser.

Alternatively, execute:

minikube service task-manager-service --url

Open the displayed URL.

Step 11: Scale the Application

Execute:

kubectl scale deployment task-manager-deployment --replicas=4

Verify:

```
kubectl get pods
```

Four application Pods should be running.

Step 12: Verify Self-Healing

Display the Pods:

```
kubectl get pods
```

Copy one Pod name.

Delete the Pod:

```
kubectl delete pod <pod-name>
```

Immediately verify:

```
kubectl get pods
```

Kubernetes automatically creates a replacement Pod to maintain the desired number of replicas.

Step 13: View Application Logs

Execute:

```
kubectl logs <pod-name>
```

Expected:

Student Task Manager running on port 3000

Step 14: Stop Minikube

After completing the experiment, execute:

```
minikube stop
```

8. Algorithm

Algorithm: Kubernetes Deployment and Scaling

1. Start.
2. Verify the Student Task Manager Docker Image.
3. Start the Minikube Cluster.
4. Verify the Kubernetes Cluster and Node.
5. Load the Docker Image into Minikube.
6. Create the Kubernetes Deployment YAML file.
7. Configure the Docker Image and Container port.
8. Configure the required number of replicas.
9. Deploy the application using kubectl.
10. Verify the Deployment.
11. Verify the application Pods.

12. Create the Kubernetes Service YAML file.
13. Configure the NodePort Service.
14. Deploy the Kubernetes Service.
15. Access the Student Task Manager application.
16. Scale the Deployment to four replicas.
17. Verify the scaled Pods.
18. Delete one application Pod.
19. Verify Kubernetes self-healing.
20. View the application Pod logs.
21. Stop the Minikube Cluster.
22. Stop.

9. Sample Input

Docker Image

task-manager:1.0

Deployment Name

task-manager-deployment

Initial Replicas

2

Scaled Replicas

4

Application Port

3000

Service Name

task-manager-service

10. Sample Output

Kubernetes Node

NAME	STATUS
------	--------

minikube	Ready
----------	-------

Initial Deployment

NAME	READY	AVAILABLE
------	-------	-----------

task-manager-deployment	2/2	2
-------------------------	-----	---

Initial Pods

task-manager-deployment-xxxxx-aaaaa	1/1	Running
-------------------------------------	-----	---------

task-manager-deployment-xxxxx-bbbbbb	1/1	Running
--------------------------------------	-----	---------

After Scaling

task-manager-deployment 4/4 4

Self-Healing

Pod Deleted

Kubernetes detected the missing Pod.

New Pod created automatically.

Desired Replicas: 4

Running Pods: 4

Application Output

Student Task Manager

Application successfully deployed on Kubernetes.

11. Outcome

After completing this experiment, students will be able to:

- Explain container orchestration.
- Understand Kubernetes architecture.
- Create Kubernetes Deployments and Services.
- Deploy a Dockerized application on Kubernetes.
- Manage application Pods.
- Scale an application using Kubernetes replicas.
- Expose an application using a NodePort Service.
- Demonstrate Kubernetes self-healing.
- Use Minikube and kubectl for local Kubernetes deployment.

12. Viva Questions

1. What is container orchestration?
2. What is Kubernetes?
3. Why is Kubernetes used in DevOps?
4. What is a Kubernetes Cluster?
5. What is the Kubernetes Control Plane?
6. What is a Worker Node?
7. What is Minikube?
8. What is kubectl?
9. What is a Kubernetes Pod?
10. What is a Kubernetes Deployment?
11. What is meant by desired state?

12. What is a Replica?
13. What is application scaling?
14. What is a Kubernetes Service?
15. Why should applications not directly use Pod IP addresses?
16. What is a NodePort Service?
17. What is the difference between port and targetPort?
18. What is Kubernetes self-healing?
19. What happens when a Pod managed by a Deployment is deleted?
20. How is Kubernetes used in an industry CI/CD workflow?



Assignment No: 6

Assignment No: 6

Title

Jenkins Pipeline with Automated Testing for a Web Application

Course Outcome (CO) Mapped: CO3

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 02

1. Aim

To configure a Jenkins Pipeline for the Student Task Manager Web Application and integrate automated test scripts to ensure that the pipeline fails automatically when test cases fail.

2. Problem Statement

Create a Jenkins Pipeline for the **Student Task Manager Web Application** stored in GitHub.

Define the CI workflow using a Jenkinsfile. Integrate pre-written automated test scripts into the pipeline and configure Jenkins to execute the tests automatically.

The assignment should demonstrate:

1. Creation of automated test scripts.
2. Configuration of a test command using npm.
3. Creation of a Jenkinsfile.
4. Creation of a Jenkins Pipeline Job.
5. Integration of GitHub with Jenkins Pipeline.
6. Automated dependency installation.
7. Automated build execution.
8. Automated test execution.
9. Successful pipeline execution when all tests pass.
10. Automatic pipeline failure when any test case fails.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand Jenkins Pipeline.
- Explain Pipeline as Code.
- Create a Jenkinsfile.
- Understand Jenkins Pipeline stages.
- Integrate automated testing into a CI workflow.
- Execute tests automatically using Jenkins.

- Analyze successful and failed pipeline executions.
- Understand fail-fast behaviour in CI pipelines.

4. Prerequisites

Students should be familiar with:

- Git and GitHub.
- Jenkins fundamentals.
- Jenkins Freestyle Projects.
- Node.js and npm.
- Basic JavaScript.
- Student Task Manager Web Application used in previous assignments.

5. Software and Hardware Requirements

Software

- Jenkins
- Java JDK supported by Jenkins
- Git
- Node.js and npm
- GitHub Account
- Visual Studio Code
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- Internet connectivity
- Minimum 5 GB free disk space

6. Theory

6.1 Continuous Integration

Continuous Integration (CI) is a DevOps practice in which developers frequently integrate source code changes into a shared repository.

Each code change is automatically validated through build and testing processes.

Developer



Push Code



GitHub



CI Pipeline



Build



Automated Tests



SUCCESS / FAILURE

CI provides faster feedback and helps identify software defects at an early stage.

6.2 Jenkins Pipeline

A Jenkins Pipeline is a sequence of automated stages used to build, test, and deliver an application.

Example:

Checkout



Install



Build



Test



Pipeline Result

Unlike a Freestyle Project, a Jenkins Pipeline can define the complete automation workflow as code.

6.3 Pipeline as Code

Pipeline as Code is the practice of defining the CI/CD workflow in a file stored with the application source code.

In Jenkins, this file is called:

Jenkinsfile

The Jenkinsfile is stored in the GitHub repository.

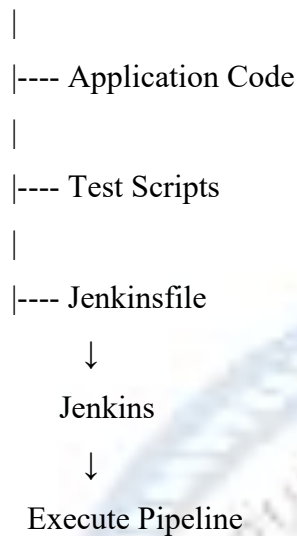
Benefits include:

- Version-controlled pipeline configuration.
- Reusable CI/CD workflows.
- Easy modification and maintenance.
- Better team collaboration.

- Consistent pipeline execution.

The workflow is:

GitHub Repository



6.4 Jenkins Pipeline Stages

A Pipeline is divided into logical stages.

For this experiment:

Checkout



Install Dependencies



Build



Automated Testing

Checkout Stage

Retrieves the latest application source code from GitHub.

Install Dependencies Stage

Executes:

```
npm install
```

This installs the dependencies defined in package.json.

Build Stage

Executes:

```
npm run build
```

The build script validates the application files.

Test Stage

Executes:

`npm test`

The automated test scripts are executed.

If any test fails, the command returns a non-zero exit code and Jenkins marks the Pipeline as failed.

6.5 Automated Testing

Automated testing uses scripts to verify application behaviour automatically.

For example, the Student Task Manager may require:

Test 1

Application file exists

Test 2

Public directory exists

Test 3

HTML page exists

Instead of manually checking the files, an automated test script performs the validation.

Example:

```
const fs = require('fs');
const tests = [
  'app.js',
  'package.json',
  'public/index.html'
];
for (const file of tests) {
  if (!fs.existsSync(file)) {
    console.error(`TEST FAILED: ${file} not found`);
    process.exit(1);
  }
  console.log(`TEST PASSED: ${file} exists`);
}
console.log('All automated tests passed.');
```

Save the file as:

`test.js`

6.6 npm Test Script

The package.json file contains application scripts.

Configure:

```
"scripts": {  
  "start": "node app.js",  
  "build": "node build.js",  
  "test": "node test.js"  
}
```

When Jenkins executes:

npm test

npm runs:

node test.js

6.7 Test Pass and Test Failure

When all test cases pass:

Test Execution

↓

All Tests Passed

↓

Exit Code 0

↓

Jenkins Pipeline SUCCESS

When a test case fails:

Test Execution

↓

Test Failed

↓

Exit Code 1

↓

Jenkins Pipeline FAILURE

This ensures that defective application code does not continue through the CI workflow.

6.8 Jenkinsfile

A Declarative Jenkins Pipeline uses the following basic structure:

```
pipeline {  
  agent any  
  stages {  
    stage('Stage Name') {
```

```
    steps {  
        // Commands  
    }  
}  
}
```

For the Student Task Manager:

```
pipeline {  
    agent any  
    stages {  
        stage('Checkout') {  
            steps {  
                checkout scm  
            }  
        }  
        stage('Install Dependencies') {  
            steps {  
                bat 'npm install'  
            }  
        }  
        stage('Build') {  
            steps {  
                bat 'npm run build'  
            }  
        }  
        stage('Automated Testing') {  
            steps {  
                bat 'npm test'  
            }  
        }  
    }  
}
```

For Linux-based Jenkins systems, use:

```
sh 'npm install'
```

sh 'npm run build'

sh 'npm test'

instead of bat.

6.9 Fail-Fast Behaviour

A Jenkins Pipeline normally stops when a stage returns an error.

Example:

Checkout → SUCCESS

Install → SUCCESS

Build → SUCCESS

Test → FAILURE

Deployment → NOT EXECUTED

This behaviour prevents failed code from moving to later stages.

In this assignment, the Pipeline should automatically fail when any automated test fails.

6.10 Freestyle Project vs Jenkins Pipeline

Freestyle Project

Configured mainly using Jenkins UI

Suitable for simple Jobs

Limited workflow visibility

Configuration stored in Jenkins

Used in Assignment No. 3

Jenkins Pipeline

Defined using Jenkinsfile

Suitable for CI/CD workflows

Stage-based visualization

Pipeline stored with source code

Used in Assignment No. 6

Jenkins Pipelines are commonly preferred for modern CI/CD workflows.

6.11 Jenkins Pipeline in Industry

A typical industry CI/CD Pipeline may contain:

Code Checkout

↓

Dependency Installation

↓

Build

↓

Unit Testing

↓

Code Quality Analysis

↓

Security Scanning



Docker Image Build



Container Registry



Kubernetes Deployment

In this experiment, the Pipeline is limited to:

Checkout



Install



Build



Automated Testing

This provides the foundation for understanding larger CI/CD pipelines.

7. Procedure

Step 1: Open the Student Task Manager Project

Verify the project structure:

devops-task-manager

```
|
|— public
|   |— index.html
|
|— app.js
|— build.js
|— test.js
|— package.json
|— Jenkinsfile
```

Step 2: Create the Automated Test Script

Create:

test.js

Add:

```
const fs = require('fs');
```

```
const tests = [  
  'app.js',  
  'package.json',  
  'public/index.html'  
];  
  
console.log('Starting automated tests...');  
for (const file of tests) {  
  if (!fs.existsSync(file)) {  
    console.error(`TEST FAILED: ${file} not found`);  
    process.exit(1);  
  }  
  console.log(`TEST PASSED: ${file} exists`);  
}  
  
console.log('All automated tests passed.');
```

Step 3: Configure the npm Test Command

Modify package.json.

```
"scripts": {  
  "start": "node app.js",  
  "build": "node build.js",  
  "test": "node test.js"  
}
```

Step 4: Execute Tests Locally

Execute:

npm test

Expected output:

TEST PASSED: app.js exists

TEST PASSED: package.json exists

TEST PASSED: public/index.html exists

All automated tests passed.

Step 5: Create the Jenkinsfile

Create:

Jenkinsfile

For Windows Jenkins:

```
pipeline {
```

```
agent any
stages {
    stage('Checkout') {
        steps {
            checkout scm
        }
    }
    stage('Install Dependencies') {
        steps {
            bat 'npm install'
        }
    }
    stage('Build') {
        steps {
            bat 'npm run build'
        }
    }
    stage('Automated Testing') {
        steps {
            bat 'npm test'
        }
    }
}
```

For Linux Jenkins, replace bat with sh.

Step 6: Push the Files to GitHub

Execute:

```
git add .
```

```
git commit -m "Add Jenkins pipeline and automated tests"
```

```
git push origin main
```

Verify that test.js and Jenkinsfile are available in the GitHub repository.

Step 7: Create a Jenkins Pipeline Job

Open the Jenkins Dashboard.

Navigate to:

New Item



Enter Job Name



Select Pipeline



OK

Job Name:

task-manager-pipeline

Step 8: Configure Pipeline from GitHub

Navigate to the **Pipeline** section.

Select:

Pipeline Script from SCM

Configure:

SCM: Git

Repository URL: GitHub Repository URL

Branch: */main

Script Path: Jenkinsfile

Save the Job.

Step 9: Execute the Pipeline

Click:

Build Now

Jenkins executes:

Checkout



Install Dependencies



Build



Automated Testing

Verify that all stages complete successfully.

Step 10: Verify Successful Pipeline

Open the Console Output.

Expected:

TEST PASSED: app.js exists

TEST PASSED: package.json exists

TEST PASSED: public/index.html exists

All automated tests passed.

Finished: SUCCESS

Step 11: Demonstrate Test Failure

Temporarily rename:

public/index.html

to:

public/index-old.html

Commit and push the change:

git add .

git commit -m "Test pipeline failure"

git push origin main

Execute the Jenkins Pipeline again.

Expected:

TEST FAILED: public/index.html not found
script returned exit code 1

Finished: FAILURE

Step 12: Restore the Application

Restore:

public/index.html

Commit and push the corrected code.

Execute the Pipeline again.

Verify:

Finished: SUCCESS

8. Algorithm

Algorithm: Jenkins Pipeline with Automated Testing

1. Start.
2. Open the Student Task Manager application.
3. Create the automated test script.
4. Define application test cases.
5. Configure the npm test command.
6. Execute the tests locally.

7. Create the Jenkinsfile.
8. Define the Checkout stage.
9. Define the Install Dependencies stage.
10. Define the Build stage.
11. Define the Automated Testing stage.
12. Push the application and Jenkinsfile to GitHub.
13. Start Jenkins.
14. Create a Jenkins Pipeline Job.
15. Configure GitHub as the SCM.
16. Configure the Jenkinsfile as the Pipeline script.
17. Execute the Jenkins Pipeline.
18. Jenkins retrieves the application source code.
19. Jenkins installs the application dependencies.
20. Jenkins executes the application build.
21. Jenkins executes the automated tests.
22. Verify the Pipeline result.
23. Introduce a test failure.
24. Execute the Pipeline again.
25. Verify automatic Pipeline failure.
26. Restore the application.
27. Execute the Pipeline successfully.
28. Stop.

9. Sample Input

Jenkins Job Name

task-manager-pipeline

Pipeline Stages

Checkout

Install Dependencies

Build

Automated Testing

Test Command

npm test

Test Files

app.js

package.json

public/index.html

10. Sample Output

Successful Pipeline

Starting automated tests...

TEST PASSED: app.js exists

TEST PASSED: package.json exists

TEST PASSED: public/index.html exists

All automated tests passed.

Finished: SUCCESS

Failed Pipeline

Starting automated tests...

TEST PASSED: app.js exists

TEST PASSED: package.json exists

TEST FAILED: public/index.html not found

script returned exit code 1

Finished: FAILURE

11. Outcome

After completing this experiment, students will be able to:

- Create a Jenkins Pipeline.
- Understand Pipeline as Code.
- Create and configure a Jenkinsfile.
- Define Jenkins Pipeline stages.
- Integrate automated testing into a CI workflow.
- Execute automated tests using Jenkins.
- Analyze successful and failed Pipeline executions.
- Understand automatic Pipeline failure when test cases fail.

12. Viva Questions

1. What is Continuous Integration?
2. What is a Jenkins Pipeline?
3. What is Pipeline as Code?
4. What is a Jenkinsfile?
5. Why is a Jenkinsfile stored in GitHub?
6. What is a Pipeline stage?

7. What is the purpose of the Checkout stage?
8. What is the purpose of npm install?
9. What is the purpose of npm test?
10. What is automated testing?
11. What happens when a test script returns exit code 1?
12. Why does Jenkins mark a Pipeline as FAILURE?
13. What is fail-fast behaviour?
14. Differentiate between a Freestyle Project and a Jenkins Pipeline.
15. How are automated tests used in an industry CI/CD Pipeline?



Assignment No: 7

Assignment No: 7

Title

Real-Time Application Monitoring Using Prometheus

Course Outcome (CO) Mapped: CO4

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 04

1. Aim

To configure Prometheus to collect and monitor real-time performance metrics of the Student Task Manager Web Application.

2. Problem Statement

Configure Prometheus to monitor the **Student Task Manager Web Application** deployed on Kubernetes.

The application should expose performance metrics through a /metrics endpoint. Prometheus should periodically collect these metrics and allow users to monitor application availability, HTTP requests, and memory usage.

The assignment should demonstrate:

1. Application metric generation.
2. Creation of a /metrics endpoint.
3. Configuration of Prometheus.
4. Deployment of Prometheus on Kubernetes.
5. Collection of application metrics.
6. Verification of Prometheus targets.
7. Execution of PromQL queries.
8. Real-time monitoring of application performance.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand application monitoring and observability.
- Explain Prometheus architecture.
- Expose application performance metrics.
- Configure Prometheus metric collection.
- Monitor Kubernetes-based applications.
- Use PromQL queries.
- Analyze application availability and performance metrics.

- Understand real-time monitoring in DevOps.

4. Prerequisites

Students should be familiar with:

- Docker and containerization.
- Kubernetes Pods, Deployments, and Services.
- Minikube and kubectl.
- Node.js and npm.
- Student Task Manager application deployed in Assignment No. 5.

5. Software and Hardware Requirements

Software

- Docker Desktop / Docker Engine
- Minikube
- kubectl
- Node.js and npm
- Prometheus
- Visual Studio Code
- Web Browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- Hardware virtualization support
- Minimum 10 GB free disk space
- Internet connectivity for initial container image download

6. Theory

6.1 Application Monitoring

Application monitoring is the process of continuously observing application health, availability, resource usage, and performance.

Important monitoring parameters include:

- Application availability.
- Number of HTTP requests.
- CPU utilization.
- Memory utilization.
- Request response time.
- Application errors.

A typical monitoring workflow is:

Application



Generate Metrics



Monitoring System



Collect Metrics



Analyze Application Performance

Monitoring helps DevOps and SRE teams identify performance problems and application failures.

6.2 Prometheus

Prometheus is an open-source monitoring and alerting system designed for collecting and storing time-series metrics.

Prometheus stores metric values together with timestamps.

Example:

task_manager_http_requests_total 125

After some time:

task_manager_http_requests_total 160

Prometheus stores these values as time-series data and allows users to analyze how metrics change over time.

Prometheus provides:

- Metric collection.
- Time-series storage.
- PromQL query language.
- Target monitoring.
- Service monitoring.
- Alerting support.

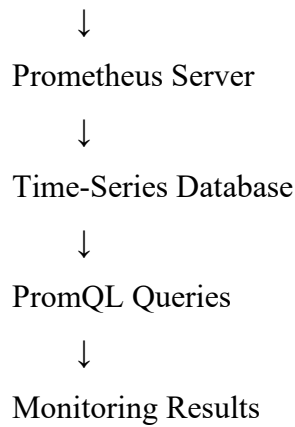
6.3 Prometheus Architecture

The basic Prometheus monitoring architecture is:

Student Task Manager



/metrics



The application exposes metrics through a /metrics endpoint.

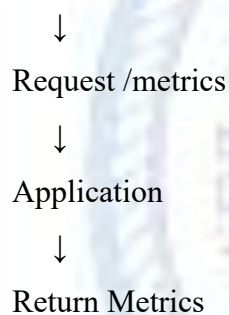
Prometheus periodically accesses the endpoint and collects the latest metric values.

This process is known as **scraping**.

6.4 Pull-Based Monitoring

Prometheus follows a **pull-based monitoring model**.

Prometheus



Prometheus periodically pulls metrics from configured targets.

The time interval between two metric collections is called the **scrape interval**.

Example:

scrape_interval: 5s

This means Prometheus collects application metrics every 5 seconds.

6.5 Prometheus Metrics

Metrics represent measurable application or system information.

Common Prometheus metric types include:

Metric Type Purpose

Counter	Continuously increasing value
Gauge	Value that can increase or decrease
Histogram	Distribution of observed values
Summary	Statistical summary of observations

In this experiment, an HTTP request Counter is used.

Example:

```
task_manager_http_requests_total
```

The Counter increases whenever the application receives an HTTP request.

6.6 Node.js Prometheus Client

The prom-client package is used to expose Node.js application metrics in Prometheus format.

Install the package using:

```
npm install prom-client
```

The package provides:

- Counters.
- Gauges.
- Histograms.
- Default Node.js metrics.

Default metrics include:

```
process_resident_memory_bytes
```

```
nodejs_heap_size_used_bytes
```

```
process_cpu_seconds_total
```

6.7 Metrics Endpoint

The application exposes metrics using:

```
/metrics
```

Example:

```
http://application:3000/metrics
```

Sample metric output:

```
# HELP task_manager_http_requests_total Total HTTP requests
```

```
# TYPE task_manager_http_requests_total counter
```

```
task_manager_http_requests_total 25
```

```
process_resident_memory_bytes 68157440
```

```
nodejs_heap_size_used_bytes 8388608
```

Prometheus collects metrics from this endpoint.

6.8 Prometheus Configuration

Prometheus uses a configuration file named:

```
prometheus.yml
```

Example:

```
global:
```

```
scrape_interval: 5s
scrape_configs:
  - job_name: 'task-manager'
    static_configs:
      - targets:
        - 'task-manager-service:3000'
```

The configuration defines:

- Scrape interval.
- Monitoring Job name.
- Application target.

The target:

task-manager-service:3000

is the Kubernetes Service created in Assignment No. 5.

6.9 Prometheus Target Status

Prometheus displays the status of configured monitoring targets.

The target may have two states:

UP

Prometheus can successfully collect metrics.

DOWN

Prometheus cannot access the metrics endpoint.

Target monitoring helps identify connectivity or application availability problems.

6.10 PromQL

PromQL is the Prometheus Query Language used to retrieve and analyze metric data.

Application availability:

```
up{job="task-manager"}
```

HTTP request counter:

```
task_manager_http_requests_total
```

HTTP request rate:

```
rate(task_manager_http_requests_total[1m])
```

Process memory:

```
process_resident_memory_bytes
```

Node.js heap memory:

```
nodejs_heap_size_used_bytes
```

PromQL queries help DevOps teams analyze application behaviour and resource utilization.

6.11 Prometheus in DevOps

A typical industry monitoring workflow is:

Applications



Expose Metrics



Prometheus



Collect and Store Metrics



PromQL



Grafana Dashboards



DevOps / SRE Teams

Prometheus focuses mainly on **metric collection and storage**.

Grafana is commonly integrated with Prometheus for advanced dashboard visualization.

Grafana visualization is covered in Assignment No. 8.

7. Procedure

Step 1: Install Prometheus Client

Open the Student Task Manager project.

Execute:

```
npm install prom-client
```

Step 2: Configure Application Metrics

Modify app.js.

Import the Prometheus client:

```
const client = require('prom-client');
```

Enable default Node.js metrics:

```
client.collectDefaultMetrics();
```

Create an HTTP request Counter:

```
const httpRequestCounter = new client.Counter({  
  name: 'task_manager_http_requests_total',  
  help: 'Total HTTP requests received'  
});
```

Step 3: Count HTTP Requests

Add the following middleware:

```
app.use((req, res, next) => {  
  httpRequestCounter.inc();  
  next();  
});
```

The Counter increases for every HTTP request.

Step 4: Create the Metrics Endpoint

Add:

```
app.get('/metrics', async (req, res) => {  
  res.set('Content-Type', client.register.contentType);  
  res.end(await client.register.metrics());  
});
```

Save the application.

Step 5: Test Metrics Locally

Start the application:

```
npm start
```

Open:

```
http://localhost:3000/metrics
```

Verify that Prometheus metrics are displayed.

Step 6: Rebuild the Docker Image

Execute:

```
docker build -t task-manager:2.0 .
```

Load the Image into Minikube:

```
minikube image load task-manager:2.0
```

Update deployment.yaml:

```
image: task-manager:2.0
```

Deploy the updated application:

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl get pods
```

Step 7: Create Prometheus Configuration

Create:

```
prometheus.yml
```

Add:

global:

scrape_interval: 5s

scrape_configs:

- job_name: 'task-manager'

static_configs:

- targets:

- 'task-manager-service:3000'

Step 8: Create Prometheus ConfigMap

Execute:

kubectl create configmap prometheus-config --from-file=prometheus.yml

Verify:

kubectl get configmap

Step 9: Create Prometheus Deployment

Create:

prometheus-deployment.yml

Add:

apiVersion: apps/v1

kind: Deployment

metadata:

name: prometheus-deployment

spec:

replicas: 1

selector:

matchLabels:

app: prometheus

template:

metadata:

labels:

app: prometheus

spec:

containers:

- name: prometheus

image: prom/prometheus:latest

ports:

- containerPort: 9090

volumeMounts:

- name: prometheus-config
mountPath: /etc/prometheus

volumes:

- name: prometheus-config

configMap:

name: prometheus-config

Step 10: Deploy Prometheus

Execute:

kubectl apply -f prometheus-deployment.yaml

Verify:

kubectl get pods

Ensure that the Prometheus Pod is:

Running

Step 11: Create Prometheus Service

Create:

prometheus-service.yaml

Add:

apiVersion: v1

kind: Service

metadata:

name: prometheus-service

spec:

type: NodePort

selector:

app: prometheus

ports:

- port: 9090

targetPort: 9090

Deploy:

kubectl apply -f prometheus-service.yaml

Step 12: Access Prometheus

Execute:

minikube service prometheus-service

The Prometheus web interface opens in the browser.

Step 13: Verify Prometheus Target

In Prometheus, navigate to:

Status



Targets

Verify:

Job: task-manager

State: UP

Step 14: Execute PromQL Queries

Execute:

```
up{job="task-manager"}
```

Then execute:

```
task_manager_http_requests_total
```

Execute:

```
rate(task_manager_http_requests_total[1m])
```

Execute:

```
process_resident_memory_bytes
```

Observe the metric values.

Step 15: Generate Application Traffic

Access the Student Task Manager application.

Refresh the application multiple times and perform application operations.

Return to Prometheus and execute:

```
task_manager_http_requests_total
```

Observe the increase in the HTTP request Counter.

8. Algorithm

Algorithm: Application Monitoring Using Prometheus

1. Start.
2. Install the Prometheus Node.js client.
3. Enable default application metrics.
4. Create an HTTP request Counter.
5. Configure HTTP request monitoring.

6. Create the /metrics endpoint.
7. Test application metrics locally.
8. Rebuild the Docker Image.
9. Deploy the updated application on Kubernetes.
10. Create the Prometheus configuration file.
11. Configure the scrape interval and application target.
12. Create the Prometheus ConfigMap.
13. Create and deploy the Prometheus Deployment.
14. Create the Prometheus Service.
15. Access the Prometheus interface.
16. Verify the application target status.
17. Execute PromQL queries.
18. Generate application traffic.
19. Observe real-time metric changes.
20. Stop.

9. Sample Input

Application Target

task-manager-service:3000

Scrape Interval

5 Seconds

Metrics Endpoint

/metrics

PromQL Queries

up{job="task-manager"}

task_manager_http_requests_total
rate(task_manager_http_requests_total[1m])

process_resident_memory_bytes

10. Sample Output

Prometheus Target Status

Job: task-manager

State: UP

HTTP Request Counter:

task_manager_http_requests_total = 45

Application Availability:

up = 1

Process Memory:

process_resident_memory_bytes = 68157440

Prometheus successfully collecting application metrics.

11. Outcome

After completing this experiment, students will be able to:

- Configure application performance metrics.
- Create a Prometheus metrics endpoint.
- Deploy Prometheus on Kubernetes.
- Configure Prometheus metric scraping.
- Monitor application target availability.
- Execute PromQL queries.
- Analyze HTTP request and memory metrics.
- Monitor application performance in real time.

12. Viva Questions

1. What is application monitoring?
2. What is Prometheus?
3. What is time-series data?
4. What is Prometheus scraping?
5. What is a scrape interval?
6. Explain the Prometheus pull-based model.
7. What is a Prometheus metric?
8. What is a Counter metric?
9. What is the purpose of the /metrics endpoint?
10. What is prom-client?
11. What is PromQL?
12. What does the up metric represent?
13. What does target status UP indicate?
14. What does target status DOWN indicate?
15. What is the purpose of the rate() function?
16. What does task_manager_http_requests_total represent?
17. What is process_resident_memory_bytes?
18. Why is a ConfigMap used for Prometheus configuration?
19. Differentiate between Prometheus and Grafana.

20. How is Prometheus used in DevOps and SRE?



Assignment No: 8

Title

Visualization of Real-Time Application Metrics Using Grafana

Course Outcome (CO) Mapped: CO4

Bloom's Taxonomy Level (BTL): L3 – Apply

Hours: 02

1. Aim

To configure Grafana dashboards for visualizing and analyzing the real-time application performance metrics collected by Prometheus.

2. Problem Statement

Configure Grafana to visualize the real-time metrics of the **Student Task Manager Web Application** collected by Prometheus.

The application monitoring environment configured in Assignment No. 7 should be used.

Perform the following tasks:

1. Deploy Grafana on Kubernetes.
2. Access the Grafana web interface.
3. Configure Prometheus as a Grafana data source.
4. Create a new Grafana dashboard.
5. Create dashboard panels using PromQL queries.
6. Visualize application availability.
7. Visualize HTTP request metrics.
8. Visualize application memory usage.
9. Visualize Node.js heap memory.
10. Generate application traffic and observe real-time dashboard updates.

3. Learning Objectives

Upon successful completion of this experiment, students will be able to:

- Understand the role of Grafana in application monitoring.
- Configure Prometheus as a Grafana data source.
- Create Grafana dashboards.
- Create and configure dashboard panels.
- Use PromQL queries in Grafana.
- Visualize real-time application metrics.
- Analyze application availability, traffic, and memory usage.
- Understand the role of dashboards in DevOps monitoring.

4. Prerequisites

Students should be familiar with:

- Prometheus fundamentals.
- Prometheus metrics.
- PromQL basics.
- Kubernetes Deployment and Services.
- Minikube and kubectl.
- Student Task Manager monitoring environment configured in Assignment No. 7.

5. Software and Hardware Requirements

Software

- Docker Desktop / Docker Engine
- Minikube
- kubectl
- Prometheus
- Grafana
- Web Browser
- Windows / Linux Operating System

Hardware

- Computer system with minimum 8 GB RAM recommended
- Hardware virtualization support
- Minimum 15 GB free disk space
- Internet connectivity for initial container image installation

6. Theory

6.1 Grafana

Grafana is an open-source visualization and monitoring platform used to create interactive dashboards from data collected by monitoring systems.

Grafana can connect to different data sources such as:

- Prometheus
- Elasticsearch
- MySQL
- PostgreSQL
- InfluxDB

In this experiment, Grafana uses **Prometheus as the data source**.

The monitoring workflow is:

Student Task Manager



Expose Metrics



Prometheus



Collect and Store Metrics



Grafana



Dashboard Visualization

Prometheus is responsible for collecting and storing metrics.

Grafana is responsible for visualizing and analyzing those metrics.

6.2 Need for Grafana

Prometheus provides a basic interface for querying and graphing metrics.

However, DevOps and SRE teams require dashboards that provide a combined view of multiple system metrics.

Example:

Application Monitoring Dashboard

|

|---- Application Status

|

|---- HTTP Request Rate

|

|---- Process Memory

|

|---- Node.js Heap Memory

Grafana provides:

- Interactive dashboards.
- Multiple visualization panels.
- Real-time metric visualization.
- Time-range selection.
- Dashboard refresh.
- PromQL query integration.

- Centralized monitoring views.

6.3 Grafana Dashboard

A Grafana Dashboard is a collection of visualization panels used to monitor a system.

Example:

Student Task Manager Dashboard

Application Status

HTTP Request Rate

Process Memory Usage

Node.js Heap Memory

Each dashboard panel displays one or more metrics.

6.4 Grafana Panel

A Panel is an individual visualization component within a Grafana Dashboard.

Common panel visualizations include:

- Time Series.
- Stat.
- Gauge.
- Bar Chart.
- Table.

For this experiment:

Metric	Visualization
Application Availability	Stat
HTTP Request Rate	Time Series
Process Memory Usage	Time Series
Node.js Heap Memory	Gauge / Time Series

6.5 Grafana Data Source

A Data Source is the system from which Grafana retrieves monitoring data.

For this experiment:

Grafana



Prometheus Data Source



Prometheus Service



Application Metrics

The Prometheus Kubernetes Service is:

prometheus-service

Prometheus runs on port:

9090

Therefore, Grafana can access Prometheus using:

<http://prometheus-service:9090>

Since Grafana and Prometheus are deployed in the same Kubernetes cluster, Kubernetes Service DNS allows Grafana to communicate with the Prometheus Service.

6.6 PromQL Queries in Grafana

Grafana uses PromQL queries to retrieve metrics from Prometheus.

Application Availability

```
up{job="task-manager"}
```

Expected value:

1 → Application Target UP

0 → Application Target DOWN

HTTP Request Rate

```
rate(task_manager_http_requests_total[1m])
```

This displays the average HTTP request rate.

Process Memory Usage

```
process_resident_memory_bytes
```

This displays the application process memory consumption.

Node.js Heap Memory

```
nodejs_heap_size_used_bytes
```

This displays the used Node.js heap memory.

6.7 Real-Time Dashboard Monitoring

Grafana periodically refreshes dashboard data.

Example:

Application Traffic



HTTP Requests Increase



Application Metrics Change



Prometheus Scrapes Metrics



Grafana Queries Prometheus



Dashboard Updated

The dashboard refresh interval can be configured.

Example:

5 seconds

10 seconds

30 seconds

The dashboard provides near real-time visualization based on the Prometheus scrape interval and Grafana refresh interval.

6.8 Grafana in DevOps

In an industry monitoring environment:

Applications



Prometheus



Grafana Dashboards



DevOps / SRE Team



Performance Analysis



Operational Decisions

Grafana dashboards help teams:

- Monitor application health.
- Analyze traffic patterns.
- Identify performance problems.
- Observe resource utilization.
- Support troubleshooting.
- Support capacity planning.

7. Procedure

Step 1: Start Minikube

Execute:

minikube start

Verify:

minikube status

Step 2: Verify Application and Prometheus

Execute:

kubectl get pods

Verify that the Student Task Manager and Prometheus Pods are running.

Execute:

kubectl get svc

Verify:

task-manager-service

prometheus-service

Step 3: Create Grafana Deployment

Create:

grafana-deployment.yaml

Add:

apiVersion: apps/v1

kind: Deployment

metadata:

name: grafana-deployment

spec:

replicas: 1

selector:

matchLabels:

app: grafana

```
template:
  metadata:
    labels:
      app: grafana
  spec:
    containers:
      - name: grafana
        image: grafana/grafana:latest
        ports:
          - containerPort: 3000
```

Step 4: Deploy Grafana

Execute:

```
kubectl apply -f grafana-deployment.yaml
```

Verify:

```
kubectl get pods
```

The Grafana Pod should display:

Running

Step 5: Create Grafana Service

Create:

```
grafana-service.yaml
```

Add:

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: grafana-service
```

```
spec:
```

```
  type: NodePort
```

```
  selector:
```

```
    app: grafana
```

```
  ports:
```

```
    - protocol: TCP
```

```
      port: 3000
```

```
      targetPort: 3000
```

Step 6: Deploy Grafana Service

Execute:

```
kubectl apply -f grafana-service.yaml
```

Verify:

```
kubectl get svc
```

Step 7: Access Grafana

Execute:

```
minikube service grafana-service
```

The Grafana Login page is displayed.

Default credentials are commonly:

Username: admin

Password: admin

Change the password if prompted.

Step 8: Configure Prometheus Data Source

In Grafana:

Connections



Data Sources



Add Data Source



Prometheus

Configure the Prometheus Server URL:

```
http://prometheus-service:9090
```

Select:

Save & Test

Verify successful data source connection.

Step 9: Create Grafana Dashboard

Navigate to:

Dashboards



New



New Dashboard



Add Visualization

Select the Prometheus data source.

Dashboard Name:

Student Task Manager Monitoring Dashboard

Step 10: Create Application Availability Panel

Enter the PromQL query:

```
up{job="task-manager"}
```

Select visualization:

Stat

Panel Title:

Application Status

Save the panel.

Step 11: Create HTTP Request Rate Panel

Create a new panel.

Enter:

```
sum(rate(task_manager_http_requests_total[1m]))
```

Select:

Time Series

Panel Title:

HTTP Request Rate

Save the panel.

Step 12: Create Process Memory Panel

Create a new panel.

Enter:

```
process_resident_memory_bytes
```

Select:

Time Series

Panel Title:

Process Memory Usage

Configure the unit as:

Bytes

Save the panel.

Step 13: Create Node.js Heap Memory Panel

Create a new panel.

Enter:

nodejs_heap_size_used_bytes

Select:

Gauge

or:

Time Series

Panel Title:

Node.js Heap Memory

Save the panel.

Step 14: Configure Dashboard Refresh

Configure the dashboard refresh interval as:

5s

or:

10s

Save the dashboard.

Step 15: Generate Application Traffic

Access the Student Task Manager application.

Refresh the application multiple times.

Perform application operations.

Example:

Open Application



Refresh Page



Add Task



Refresh Page

Step 16: Observe Grafana Dashboard

Observe:

- Application Status.
- HTTP Request Rate.
- Process Memory Usage.
- Node.js Heap Memory.

Verify that the dashboard panels update automatically.

8. Algorithm

Algorithm: Real-Time Metrics Visualization Using Grafana

1. Start.
2. Start the Minikube cluster.
3. Verify the Student Task Manager application.
4. Verify the Prometheus monitoring service.
5. Create the Grafana Deployment.
6. Deploy Grafana on Kubernetes.
7. Create the Grafana Service.
8. Access the Grafana web interface.
9. Log in to Grafana.
10. Add Prometheus as a data source.
11. Configure the Prometheus Service URL.
12. Verify the Prometheus connection.
13. Create a Grafana Dashboard.
14. Create the Application Status panel.
15. Configure the application availability PromQL query.
16. Create the HTTP Request Rate panel.
17. Configure the request rate PromQL query.
18. Create the Process Memory panel.
19. Configure the process memory PromQL query.
20. Create the Node.js Heap Memory panel.
21. Configure the heap memory PromQL query.
22. Configure the dashboard refresh interval.
23. Save the dashboard.
24. Generate application traffic.
25. Observe the real-time dashboard updates.
26. Analyze application performance metrics.
27. Stop.

9. Sample Input

Dashboard Name

Student Task Manager Monitoring Dashboard

Prometheus Data Source URL

<http://prometheus-service:9090>

PromQL Queries

```
up{job="task-manager"}  
sum(rate(task_manager_http_requests_total[1m]))  
process_resident_memory_bytes  
nodejs_heap_size_used_bytes
```

Dashboard Refresh Interval

5 Seconds

10. Sample Output

Student Task Manager Monitoring Dashboard

Application Status

UP

HTTP Request Rate

0.35 requests/second

Process Memory Usage

65 MB

Node.js Heap Memory

8.2 MB

Dashboard Refresh

5 Seconds

Grafana Dashboard Successfully Configured

The dashboard displays real-time application metrics collected by Prometheus.

11. Outcome

After completing this experiment, students will be able to:

- Configure Grafana on Kubernetes.
- Integrate Prometheus with Grafana.
- Create Grafana dashboards.
- Create visualization panels.
- Apply PromQL queries for metric visualization.
- Monitor application availability.
- Visualize HTTP request traffic.
- Monitor application memory usage.
- Analyze real-time application performance metrics.

12. Viva Questions

1. What is Grafana?

2. What is the role of Grafana in DevOps?
3. Why is Grafana used with Prometheus?
4. Differentiate between Prometheus and Grafana.
5. What is a Grafana Dashboard?
6. What is a Grafana Panel?
7. What is a Grafana Data Source?
8. Which data source is used in this experiment?
9. What is PromQL?
10. How does Grafana retrieve metrics from Prometheus?
11. What does the up metric represent?
12. What does up = 1 indicate?
13. What does up = 0 indicate?
14. What is the purpose of the rate() function?
15. What does task_manager_http_requests_total represent?
16. What is a Time Series visualization?
17. What is a Stat visualization?
18. What is a Gauge visualization?
19. Which visualization is used for application status?
20. Which visualization is suitable for request rate?
21. What is process_resident_memory_bytes?
22. What is nodejs_heap_size_used_bytes?
23. What is a dashboard refresh interval?
24. How does Grafana provide real-time visualization?
25. Why is Prometheus configured as a Grafana data source?
26. Why is prometheus-service:9090 used?
27. How does Kubernetes Service DNS help Grafana communicate with Prometheus?
28. What happens to the dashboard when application traffic increases?
29. How are Grafana dashboards used by DevOps teams?
30. Explain the complete monitoring workflow from application to Grafana.